Module 11: Activity Response Correlator Implementation Plan

> ***For agentic workers:*** *REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.*

Goal: Implement Module 11 (per-activity glucose/HR/BP correlation with exercise session windows) and Module 11b (weekly fitness pattern aggregation with VO2max estimation and exercise dose-response analysis) as two separate Flink jobs.

Architecture: Exercise-session-window-driven KeyedProcessFunction where an activity event OPENS a window, HR/glucose/BP readings FILL it during exercise AND a 2h recovery phase, and a processing-time timer CLOSES it at `activity_duration + 2h + 5min`. Three-phase HR analysis (pre-exercise baseline, active exercise, recovery). Glucose exercise analysis distinguishes aerobic (glucose drop) vs anaerobic (catecholamine spike). Module 11b runs weekly aggregation with submaximal VO2max estimation, MET-minutes dose-response, and fitness level progression tracking. Both jobs keyed by `patientId`, consuming from `enriched-patient-events-v1`.

Tech Stack: Flink 2.1.0, Java 17, Jackson 2.17, Kafka (Confluent Cloud), JUnit 5

Clinical Foundation

Heart Rate Recovery (HRR) — Prognostic Marker

Heart rate recovery is the rate at which heart rate falls after exercise cessation. Delayed HRR reflects impaired vagal reactivation and autonomic dysfunction. Cole et al. (NEJM 1999) established that $HRR_1 < 12$ bpm (1-minute post-peak) is an independent predictor of all-cause mortality (RR 2.0, adjusted for age, fitness, and standard risk factors). Shetler et al. (JACC 2001) extended this: $HRR_2 < 22$ bpm at 2 minutes is similarly prognostic. The mechanism is parasympathetic withdrawal — healthy individuals exhibit rapid vagal reactivation post-exercise, producing a steep HR decline.

Classification thresholds used in this module: - EXCELLENT: $HRR_1 \geq 25$ bpm — strong vagal tone, high cardiorespiratory fitness - NORMAL: HRR_1 18–24 bpm — adequate parasympathetic reactivation - BLUNTED: HRR_1 12–17 bpm — borderline autonomic function, warrants monitoring - ABNORMAL: $HRR_1 <$ 12 bpm — impaired vagal reactivation, independent mortality predictor - INSUFFICIENT_DATA: Fewer than 2 post-exercise HR readings within 2 minutes of peak

Exercise-Induced Glucose Dynamics

Exercise modulates glucose through competing mechanisms. During moderate aerobic exercise (40–70% VO2max), GLUT4 transporter translocation increases skeletal muscle glucose uptake 5–50-fold independently of insulin (Richter & Hargreaves, Physiol Rev 2013). Hepatic glucose output initially compensates, but in patients with T2DM or insulin-treated T1DM, the imbalance can produce exercise-induced hypoglycemia (glucose < 70 mg/dL).

During high-intensity exercise (>80% VO2max), catecholamine release (epinephrine, norepinephrine) stimulates hepatic glycogenolysis and gluconeogenesis, producing a transient glucose spike of 20–60 mg/dL that peaks 15–30 minutes post-exercise and resolves over 1–2 hours. This is the "anaerobic glucose paradox" — intense exercise raises glucose despite increased peripheral uptake.

Late-onset post-exercise hypoglycemia (LEH) can occur 6–15 hours post-exercise due to ongoing non-insulin-mediated glucose uptake for glycogen resynthesis. While this module's 2h recovery window won't capture LEH, the glucose nadir trend from Module 11b's weekly aggregation can flag patients at risk.

Key glucose metrics for exercise sessions: - Pre-exercise baseline (last glucose reading within 30 min before activity) - Exercise glucose delta (mean during exercise minus baseline) - Glucose nadir (minimum during activity + 1h post) - Post-exercise glucose at 30/60/120 min recovery - Hypoglycemia flag (any reading < 70 mg/dL) - Rebound hyperglycemia flag (any reading > baseline + 40 mg/dL post-exercise)

Exercise Blood Pressure Response

Normal SBP rises 20–40 mmHg during moderate dynamic exercise and should return to within 10 mmHg of pre-exercise within 6 minutes of cessation. An exaggerated BP response (SBP > 210 mmHg in men, > 190 mmHg in women, or rise > 60 mmHg) is an independent predictor of future hypertension (Miyai et al., Hypertension 2000) and cardiovascular events (Schultz et al., JACC 2013).

Post-exercise hypotension (PEH) — a sustained SBP reduction of 5–7 mmHg below pre-exercise levels lasting 12–16 hours — is clinically beneficial in hypertensive patients and is evidence of intact vascular autoregulation. Kenney & Seals (Hypertension 1993) demonstrated that a single bout of moderate exercise produces PEH magnitude proportional to resting BP.

BP classification for exercise sessions: - NORMAL: SBP rise ≤ 60 mmHg, peak SBP < 210/190 (M/F) - EXAGGERATED: SBP rise > 60 mmHg or peak SBP ≥ 210/190 - HYPOTENSIVE: Post-exercise SBP drops > 20 mmHg below pre-exercise (orthostatic risk) - POST_EXERCISE_HYPOTENSION: Post-exercise SBP 5–20 mmHg below pre-exercise (beneficial) - INCOMPLETE: Missing pre or post readings

Rate-Pressure Product (RPP)

RPP = HR × SBP is a validated noninvasive proxy for myocardial oxygen demand (MVO_2). Gobel et al. (Circulation 1978) showed RPP correlates (r=0.88) with directly measured MVO_2 during exercise. Resting RPP of 6,000–10,000 is normal; peak exercise RPP of 25,000–40,000 is typical. RPP > 40,000 suggests excessive cardiac workload.

Metabolic Equivalents (METs) and Exercise Dose

One MET = 3.5 mL O_2/kg/min (resting metabolic rate). Exercise intensity is expressed in METs: walking 3 mph ≈ 3.3 METs, running 6 mph ≈ 10 METs, cycling 150W ≈ 7 METs. The Compendium of Physical Activities (Ainsworth et al., Med Sci Sports Exerc 2011) provides standardized MET values for >800 activities.

MET-minutes = METs × duration_minutes is the standard exercise dose metric. WHO guidelines recommend 150–300 MET-minutes/week of moderate activity or 75–150 MET-minutes/week of vigorous activity for cardiovascular benefit.

VO2max Estimation from Submaximal HR

VO2max (maximal oxygen uptake) is the gold-standard measure of cardiorespiratory fitness. Direct measurement requires graded exercise testing with gas exchange analysis. However, submaximal estimation is clinically practical:

Borg-derived estimation (simplified Astrand-Ryhming): VO_2max ≈ 15 × (HR_max / HR_rest). Requires known resting HR and observed peak HR. Accuracy ±10–15%.

ACSM submaximal formula: For steady-state exercise at known workload, VO_2max = VO_2_submaximal × (HR_max_predicted - HR_rest) / (HR_exercise - HR_rest). This uses the linear relationship between HR and VO_2 to extrapolate to maximal capacity.

Fitness classification (ACSM, by age-adjusted percentile): - EXCELLENT: ≥95th percentile - GOOD: 75th–94th percentile
- AVERAGE: 25th–74th percentile - BELOW_AVERAGE: 5th–24th percentile - POOR: <5th percentile

---

File Structure

Source Files (16 files)

- Create: `models/ActivityIntensityZone.java` — enum: 5 HR training zones
- Create: `models/ExerciseType.java` — enum: AEROBIC/RESISTANCE/HIIT/FLEXIBILITY/MIXED
- Create: `models/HRRecoveryClass.java` — enum: 5 HRR classifications
- Create: `models/ExerciseBPResponse.java` — enum: 5 BP response classifications
- Create: `models/FitnessLevel.java` — enum: 5 fitness classifications
- Create: `models/HRWindow.java` — timestamped HR readings buffer with zone tracking
- Create: `models/ActivityCorrelationState.java` — per-patient session state for Module 11
- Create: `models/ActivityResponseRecord.java` — output model (32 fields)
- Create: `models/FitnessPatternSummary.java` — output model (24 fields)
- Create: `models/FitnessPatternState.java` — per-patient weekly aggregation state

- Create: `operators/Module11HRAnalyzer.java` — HR features: peak, mean, HRR_1, HRR_2, zone distribution, RPP
- Create: `operators/Module11GlucoseExerciseAnalyzer.java` — exercise glucose delta, nadir, rebound, hypo flag
- Create: `operators/Module11ExerciseBPAnalyzer.java` — exercise BP response classification
- Create: `operators/Module11_ActivityResponseCorrelator.java` — main KPF with exercise session windows
- Create: `operators/Module11b_FitnessPatternAggregator.java` — weekly aggregation KPF
- Create: `operators/Module11bVO2maxEstimator.java` — submaximal VO2max estimation
- Create: `operators/Module11bExerciseDoseCalculator.java` — MET-minutes dose-response
- Modify: `FlinkJobOrchestrator.java` — add module11/module11b cases + launch methods

Test Files (9 files)

- Create: `builders/Module11TestBuilder.java` — event + state factories
- Create: `operators/Module11HRAnalyzerTest.java`
- Create: `operators/Module11GlucoseExerciseAnalyzerTest.java`
- Create: `operators/Module11ExerciseBPAnalyzerTest.java`
- Create: `operators/Module11SessionWindowTest.java`
- Create: `operators/Module11ConcurrentActivityTest.java`
- Create: `operators/Module11IntensityZoneTest.java`
- Create: `operators/Module11bVO2maxEstimatorTest.java`
- Create: `operators/Module11bExerciseDoseCalculatorTest.java`

Task 1: Create Model Enums

Files: - Create: `models/ActivityIntensityZone.java` - Create: `models/ExerciseType.java` - Create: `models/HRRecoveryClass.java` - Create: `models/ExerciseBPResponse.java` - Create: `models/FitnessLevel.java`

- [ ] **Step 1: Create ActivityIntensityZone enum**

```
package com.cardiofit.flink.models;

/**
 * Heart rate training zone classification based on percentage of
 * age-predicted maximum heart rate (HR_max = 220 - age).
 *
 * Five-zone model aligned with ACSM guidelines and Karvonen formula.
 * Zone boundaries use % of HR_max (not HR reserve) for simplicity
 * in streaming computation where resting HR may be unavailable.
 *
```

```
 * Clinical significance:
 * - Zones 1-2: fat oxidation predominant, safe for cardiac rehab patients
 * - Zone 3: lactate threshold transition, maximal steady-state
 * - Zone 4-5: anaerobic, catecholamine-driven glucose spike expected
 */
public enum ActivityIntensityZone {

    ZONE_1_RECOVERY,   // 50–59% HR_max — warm-up, cool-down, very light
    ZONE_2_AEROBIC,    // 60–69% HR_max — base endurance, fat oxidation peak
    ZONE_3_TEMPO,      // 70–79% HR_max — lactate threshold, "comfortably hard"
    ZONE_4_THRESHOLD,  // 80–89% HR_max — anaerobic threshold, glycolytic
    ZONE_5_ANAEROBIC;  // ≥90% HR_max — VO2max effort, catecholamine surge

    /** Default HR_max estimation when age unknown (conservative for safety). */
    public static final double DEFAULT_HR_MAX = 180.0;

    /**
     * Classify a heart rate into a training zone.
     *
     * @param heartRate current HR in bpm
     * @param hrMax     age-predicted HR_max (220 - age), or DEFAULT_HR_MAX
     * @return the intensity zone
     */
    public static ActivityIntensityZone fromHeartRate(double heartRate, double hrMax) {
        if (hrMax <= 0) hrMax = DEFAULT_HR_MAX;
        double pct = heartRate / hrMax;
        if (pct < 0.50) return ZONE_1_RECOVERY; // below zone 1 still maps to recovery
        if (pct < 0.60) return ZONE_1_RECOVERY;
        if (pct < 0.70) return ZONE_2_AEROBIC;
        if (pct < 0.80) return ZONE_3_TEMPO;
        if (pct < 0.90) return ZONE_4_THRESHOLD;
        return ZONE_5_ANAEROBIC;
    }

    /**
     * Compute age-predicted HR_max using Tanaka formula (more accurate than 220-age
     * for older adults): HR_max = 208 - 0.7 × age.
     * Falls back to 220-age formula if age <= 0.
     */
    public static double estimateHRMax(int age) {
        if (age <= 0) return DEFAULT_HR_MAX;
        return 208.0 - 0.7 * age;
    }

    /** Whether this zone is considered high-intensity (catecholamine-driven glucose spike expected). */
    public boolean isHighIntensity() {
        return this == ZONE_4_THRESHOLD || this == ZONE_5_ANAEROBIC;
    }

    /** Whether this zone is predominantly aerobic (glucose drop expected). */
    public boolean isAerobic() {
        return this == ZONE_1_RECOVERY || this == ZONE_2_AEROBIC || this == ZONE_3_TEMPO;
```

```
    }
}
```

- [ ] **Step 2: Create ExerciseType enum**

```
package com.cardiofit.flink.models;

/**
 * Exercise modality classification.
 *
 * Determines expected glucose response pattern and MET estimation approach.
 * - AEROBIC: continuous rhythmic large-muscle activity (walking, running, cycling, swimming)
 *   Expected: glucose drops 20–60 mg/dL over 30–60 min due to GLUT4 translocation
 * - RESISTANCE: weight lifting, bodyweight exercises, resistance bands
 *   Expected: transient glucose spike (10–30 mg/dL) from catecholamines + isometric BP rise
 * - HIIT: alternating high/low intensity intervals
 *   Expected: glucose spike during work intervals, net drop during recovery
 * - FLEXIBILITY: yoga, stretching, Pilates, tai chi
 *   Expected: minimal glucose change (<10 mg/dL), HR stays in zone 1-2
 * - MIXED: circuit training or unclassified combination
 *   Expected: variable — treated as moderate aerobic for MET estimation
 *
 * MET reference values from Compendium of Physical Activities (Ainsworth et al., 2011).
 */
public enum ExerciseType {

    AEROBIC,       // Running, cycling, swimming, walking, rowing, elliptical
    RESISTANCE,    // Weight lifting, bodyweight, resistance bands
    HIIT,          // Interval training, Tabata, CrossFit-style
    FLEXIBILITY,   // Yoga, stretching, Pilates, tai chi
    MIXED;         // Circuit training, sports, unclassified

    /**
     * Default MET value for moderate intensity within each exercise type.
     * Used when no specific MET value provided in the event payload.
     * These are conservative midpoint estimates.
     */
    public double getDefaultMETs() {
        switch (this) {
            case AEROBIC:     return 6.0;  // moderate jogging/cycling
            case RESISTANCE:  return 5.0;  // moderate weight training
            case HIIT:        return 8.0;  // high-intensity intervals
            case FLEXIBILITY: return 2.5;  // moderate yoga
            case MIXED:       return 5.5;  // moderate circuit
            default:          return 4.0;
        }
    }

    /**
     * Whether this exercise type typically produces a catecholamine-driven glucose spike.
     * Used by Module11GlucoseExerciseAnalyzer to set expected direction.
     */
    public boolean expectsGlucoseSpike() {
```

```
        return this == RESISTANCE || this == HIIT;
    }

    public static ExerciseType fromString(String type) {
        if (type == null) return MIXED;
        switch (type.toUpperCase().trim().replace(" ", "_")) {
            case "AEROBIC":
            case "CARDIO":
            case "RUNNING":
            case "CYCLING":
            case "SWIMMING":
            case "WALKING":
                return AEROBIC;
            case "RESISTANCE":
            case "WEIGHTS":
            case "STRENGTH":
            case "WEIGHT_TRAINING":
                return RESISTANCE;
            case "HIIT":
            case "INTERVAL":
            case "INTERVALS":
            case "TABATA":
            case "CROSSFIT":
                return HIIT;
            case "FLEXIBILITY":
            case "YOGA":
            case "STRETCHING":
            case "PILATES":
            case "TAI_CHI":
                return FLEXIBILITY;
            default:
                return MIXED;
        }
    }
}
```

- [ ] **Step 3: Create HRRecoveryClass enum**

```
package com.cardiofit.flink.models;

/**
 * Heart rate recovery classification based on HRR₁ (1-minute post-peak drop).
 *
 * Clinical basis: Cole et al. (NEJM 1999, n=2,428) showed HRR₁ < 12 bpm
 * independently predicts all-cause mortality (RR 2.0, 95% CI 1.5–2.7).
 * Shetler et al. (JACC 2001, n=2,193) validated HRR₂ < 22 bpm.
 *
 * This module uses HRR₁ as the primary classifier with the following thresholds:
 * - EXCELLENT ≥ 25 bpm: strong vagal tone, athletes and highly fit individuals
 * - NORMAL 18–24 bpm: adequate parasympathetic reactivation
 * - BLUNTED 12–17 bpm: borderline, merits longitudinal tracking
 * - ABNORMAL < 12 bpm: impaired vagal reactivation, independent mortality risk
 * - INSUFFICIENT_DATA: <2 HR readings within 2 min post-peak
```

```
 *
 * Note: These thresholds assume active recovery (continued walking/light movement).
 * Passive recovery (supine rest) produces ~6 bpm higher HRR₁ values.
 */
public enum HRRecoveryClass {

    EXCELLENT,          // HRR₁ ≥ 25 bpm
    NORMAL,             // HRR₁ 18–24 bpm
    BLUNTED,            // HRR₁ 12–17 bpm
    ABNORMAL,           // HRR₁ < 12 bpm — prognostic flag
    INSUFFICIENT_DATA;  // <2 post-exercise HR readings within 2 min

    /** Minimum post-exercise HR readings required for classification. */
    public static final int MIN_RECOVERY_READINGS = 2;

    /** Recovery window: 2 minutes post-peak in which to measure HRR₁. */
    public static final long RECOVERY_WINDOW_MS = 2L * 60_000L;

    /**
     * Classify from 1-minute heart rate recovery value.
     * @param hrr1 drop in bpm from peak HR to HR at ~1 minute post-exercise
     * @return HRRecoveryClass
     */
    public static HRRecoveryClass fromHRR1(double hrr1) {
        if (hrr1 >= 25.0) return EXCELLENT;
        if (hrr1 >= 18.0) return NORMAL;
        if (hrr1 >= 12.0) return BLUNTED;
        return ABNORMAL;
    }

    /**
     * Whether this classification warrants clinical attention.
     * ABNORMAL is a red flag; BLUNTED warrants monitoring.
     */
    public boolean isPrognosticFlag() {
        return this == ABNORMAL;
    }
}
```

- [ ] **Step 4: Create ExerciseBPResponse enum**

```
package com.cardiofit.flink.models;

/**
 * Blood pressure response classification during and after exercise.
 *
 * Clinical basis:
 * - NORMAL: SBP rise ≤ 60 mmHg and peak SBP < 210 (men) / 190 (women)
 *   DBP stays flat or drops slightly during dynamic exercise.
 *
 * - EXAGGERATED: SBP rise > 60 mmHg or peak SBP ≥ 210/190 mmHg.
 *   Miyai et al. (Hypertension 2000, n=6,578) showed exaggerated exercise BP
 *   independently predicts future hypertension (HR 1.7 over 5 years).
```

```
 *   Schultz et al. (JACC 2013) linked it to LV hypertrophy and CV events.
 *
 * - HYPOTENSIVE_RESPONSE: Post-exercise SBP drops > 20 mmHg below pre-exercise.
 *   Suggests autonomic dysfunction or excessive vasodilation. Orthostatic risk.
 *
 * - POST_EXERCISE_HYPOTENSION: Post-exercise SBP drops 5–20 mmHg below pre-exercise.
 *   Clinically beneficial, especially in hypertensive patients.
 *   Kenney & Seals (Hypertension 1993) showed PEH magnitude correlates with resting BP.
 *
 * - INCOMPLETE: Missing pre-exercise or post-exercise BP reading.
 *
 * Note: This module uses a single threshold (210 mmHg) for both sexes unless
 * sex is available in patient metadata. When sex is available, use 190 mmHg for female.
 */
public enum ExerciseBPResponse {

    NORMAL,                       // Expected rise, normal peak
    EXAGGERATED,                  // SBP rise > 60 mmHg or peak ≥ 210
    HYPOTENSIVE_RESPONSE,         // Post-exercise SBP drop > 20 mmHg below pre
    POST_EXERCISE_HYPOTENSION,    // Post-exercise SBP drop 5–20 mmHg below pre (beneficial)
    INCOMPLETE;                   // Missing BP readings

    public static final double EXAGGERATED_RISE_THRESHOLD = 60.0;     // mmHg
    public static final double EXAGGERATED_PEAK_THRESHOLD = 210.0;    // mmHg (male default)
    public static final double HYPOTENSIVE_DROP_THRESHOLD = -20.0;    // mmHg (post - pre)
    public static final double PEH_DROP_THRESHOLD = -5.0;             // mmHg

    /**
     * Classify exercise BP response from pre/peak/post readings.
     *
     * @param preSBP    SBP before exercise (within 30 min prior), or null
     * @param peakSBP   highest SBP during exercise, or null
     * @param postSBP   SBP during recovery (5–15 min post-exercise), or null
     * @return classification
     */
    public static ExerciseBPResponse classify(Double preSBP, Double peakSBP, Double postSBP) {
        if (preSBP == null || (peakSBP == null && postSBP == null)) {
            return INCOMPLETE;
        }

        // Check exaggerated response (during exercise)
        if (peakSBP != null) {
            double rise = peakSBP - preSBP;
            if (rise > EXAGGERATED_RISE_THRESHOLD || peakSBP >= EXAGGERATED_PEAK_THRESHOLD) {
                return EXAGGERATED;
            }
        }

        // Check post-exercise response
        if (postSBP != null) {
            double postDelta = postSBP - preSBP;
            if (postDelta < HYPOTENSIVE_DROP_THRESHOLD) {
                return HYPOTENSIVE_RESPONSE;
            }
```

```
            if (postDelta < PEH_DROP_THRESHOLD) {
                return POST_EXERCISE_HYPOTENSION;
            }
        }

        return NORMAL;
    }

    public boolean isPrognosticFlag() {
        return this == EXAGGERATED || this == HYPOTENSIVE_RESPONSE;
    }
}
```

- [ ] **Step 5: Create FitnessLevel enum**

```
package com.cardiofit.flink.models;

/**
 * Cardiorespiratory fitness classification based on estimated VO2max.
 *
 * Thresholds derived from ACSM's Guidelines for Exercise Testing and Prescription
 * (11th ed., 2021), stratified by age and sex. This module uses the following
 * simplified age-sex-pooled thresholds (mL/kg/min):
 *
 * EXCELLENT: VO2max ≥ 45 — top quintile, consistent training
 * GOOD:     VO2max 35–44 — above average, regular activity
 * AVERAGE:  VO2max 25–34 — sedentary to moderately active
 * BELOW_AVERAGE: VO2max 18–24 — deconditioned, cardiac rehab range
 * POOR:     VO2max < 18 — severely deconditioned, functional limitation
 * INSUFFICIENT_DATA: fewer than 3 exercise sessions for estimation
 *
 * Clinical significance: Each 1 MET (3.5 mL/kg/min) increase in fitness
 * is associated with 13–15% reduction in all-cause mortality
 * (Kodama et al., JAMA 2009, meta-analysis n=102,980).
 */
public enum FitnessLevel {

    EXCELLENT,          // VO2max ≥ 45 mL/kg/min
    GOOD,               // VO2max 35–44
    AVERAGE,            // VO2max 25–34
    BELOW_AVERAGE,      // VO2max 18–24
    POOR,               // VO2max < 18
    INSUFFICIENT_DATA;  // <3 exercise sessions

    public static final int MIN_SESSIONS_FOR_ESTIMATION = 3;

    public static FitnessLevel fromVO2max(double vo2max) {
        if (vo2max >= 45.0) return EXCELLENT;
        if (vo2max >= 35.0) return GOOD;
        if (vo2max >= 25.0) return AVERAGE;
        if (vo2max >= 18.0) return BELOW_AVERAGE;
        return POOR;
    }
```

```
    /** 1 MET = 3.5 mL/kg/min */
    public static FitnessLevel fromMETs(double mets) {
        return fromVO2max(mets * 3.5);
    }
}
```

- [ ] **Step 6: Compile enums**

Run: `cd backend/shared-infrastructure/flink-processing && mvn compile -pl . -q 2>&1 | tail -5`
Expected: BUILD SUCCESS

- [ ] **Step 7: Commit**

```
git add \
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/ActivityIntensi
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/ExerciseType.ja
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/HRRecoveryClass
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/ExerciseBPRespo
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/FitnessLevel.ja
git commit -m "feat(module11): add ActivityIntensityZone, ExerciseType, HRRecoveryClass, ExerciseBPRespon
```

Task 2: Create HRWindow Model

Files: - Create: `models/HRWindow.java`

- [ ] **Step 1: Create HRWindow**

```
package com.cardiofit.flink.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;

/**
 * Timestamped heart rate readings buffer for a single exercise session window.
 *
 * Three-phase structure:
 * 1. Pre-exercise: readings within 10 min before activity start (baseline HR)
 * 2. Active exercise: readings during the activity (peak HR, zone distribution)
 * 3. Recovery: readings within 2h after activity end (HRR₁, HRR₂)
 *
 * HR sources: wearable HR monitor (most accurate), smartwatch optical HR,
 * or manually entered resting HR. Source reliability is tracked for quality scoring.
```

```java
 *
 * Zone distribution is computed relative to age-predicted HR_max.
 */
@JsonIgnoreProperties(ignoreUnknown = true)
public class HRWindow implements Serializable {

    private static final long serialVersionUID = 1L;

    @JsonProperty("readings")
    private List<HRReading> readings;

    @JsonProperty("restingHR")
    private Double restingHR;

    @JsonProperty("hrMax")
    private double hrMax;

    @JsonProperty("activityStartTime")
    private long activityStartTime;

    @JsonProperty("activityEndTime")
    private Long activityEndTime;

    @JsonProperty("windowCloseTime")
    private long windowCloseTime;

    public HRWindow() {
        this.readings = new ArrayList<>();
        this.hrMax = ActivityIntensityZone.DEFAULT_HR_MAX;
    }

    public void addReading(long timestamp, double heartRate, String source) {
        readings.add(new HRReading(timestamp, heartRate, source));
    }

    public int size() {
        return readings.size();
    }

    public boolean isEmpty() {
        return readings.isEmpty();
    }

    public void sortByTime() {
        readings.sort((a, b) -> Long.compare(a.timestamp, b.timestamp));
    }

    /**
     * Get readings during the active exercise phase only.
     * Returns readings between activityStartTime and activityEndTime.
     */
    public List<HRReading> getActivePhaseReadings() {
        List<HRReading> active = new ArrayList<>();
        long end = activityEndTime != null ? activityEndTime : windowCloseTime;
```

```
        for (HRReading r : readings) {
            if (r.timestamp >= activityStartTime && r.timestamp <= end) {
                active.add(r);
            }
        }
        return active;
    }

    /**
     * Get readings during the recovery phase (after activity end).
     */
    public List<HRReading> getRecoveryPhaseReadings() {
        if (activityEndTime == null) return new ArrayList<>();
        List<HRReading> recovery = new ArrayList<>();
        for (HRReading r : readings) {
            if (r.timestamp > activityEndTime) {
                recovery.add(r);
            }
        }
        return recovery;
    }

    /**
     * Compute time-in-zone distribution (in milliseconds per zone).
     * Uses trapezoidal interpolation between readings to estimate zone occupancy.
     */
    public Map<ActivityIntensityZone, Long> computeZoneDistribution() {
        Map<ActivityIntensityZone, Long> zones = new EnumMap<>(ActivityIntensityZone.class);
        for (ActivityIntensityZone z : ActivityIntensityZone.values()) {
            zones.put(z, 0L);
        }
        List<HRReading> active = getActivePhaseReadings();
        if (active.size() < 2) return zones;
        for (int i = 0; i < active.size() - 1; i++) {
            double avgHR = (active.get(i).heartRate + active.get(i + 1).heartRate) / 2.0;
            long dt = active.get(i + 1).timestamp - active.get(i).timestamp;
            ActivityIntensityZone zone = ActivityIntensityZone.fromHeartRate(avgHR, hrMax);
            zones.merge(zone, dt, Long::sum);
        }
        return zones;
    }

    // --- Getters/Setters ---
    public List<HRReading> getReadings() { return readings; }
    public Double getRestingHR() { return restingHR; }
    public void setRestingHR(Double v) { this.restingHR = v; }
    public double getHrMax() { return hrMax; }
    public void setHrMax(double v) { this.hrMax = v; }
    public long getActivityStartTime() { return activityStartTime; }
    public void setActivityStartTime(long v) { this.activityStartTime = v; }
    public Long getActivityEndTime() { return activityEndTime; }
    public void setActivityEndTime(Long v) { this.activityEndTime = v; }
    public long getWindowCloseTime() { return windowCloseTime; }
    public void setWindowCloseTime(long v) { this.windowCloseTime = v; }
```

```
    @JsonIgnoreProperties(ignoreUnknown = true)
    public static class HRReading implements Serializable {
        private static final long serialVersionUID = 1L;

        @JsonProperty("timestamp")
        public long timestamp;

        @JsonProperty("heartRate")
        public double heartRate;

        @JsonProperty("source")
        public String source; // "WEARABLE", "SMARTWATCH", "MANUAL"

        public HRReading() {}

        public HRReading(long timestamp, double heartRate, String source) {
            this.timestamp = timestamp;
            this.heartRate = heartRate;
            this.source = source;
        }
    }
}
```

- [ ] **Step 2: Compile**

Run: `cd backend/shared-infrastructure/flink-processing && mvn compile -pl . -q 2>&1 | tail -5`
Expected: BUILD SUCCESS

- [ ] **Step 3: Commit**

```
git add backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/HRWindow.
git commit -m "feat(module11): add HRWindow with three-phase HR buffer and zone distribution"
```

Task 3: Create ActivityCorrelationState

Files: - Create: `models/ActivityCorrelationState.java`

- [ ] **Step 1: Create ActivityCorrelationState**

This is the per-patient keyed state for Module 11. It tracks open activity sessions, the last known resting HR, and patient age for HR_max estimation.

```
package com.cardiofit.flink.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;
import java.io.Serializable;
```

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Per-patient state for Module 11 Activity Response Correlator.
 *
 * Key design decisions:
 * - activeSessions: Map<activityEventId, ActivitySession> — supports concurrent activities
 *   (e.g., patient logs walk + wearable detects cycling, though rare)
 * - lastRestingHR: most recent resting HR for HRR calculation and VO2max estimation
 * - patientAge: for HR_max estimation (Tanaka formula: 208 - 0.7 × age)
 * - lastBPReading: retroactive buffer for pre-exercise BP (most recent within 30 min)
 * - Session window: activity_duration + 2h recovery + 5 min grace
 * - Max window cap: 6h05m (prevents runaway timers from unbounded activity durations)
 *
 * State TTL: 7 days (OnReadAndWrite + NeverReturnExpired).
 */
@JsonIgnoreProperties(ignoreUnknown = true)
public class ActivityCorrelationState implements Serializable {

    private static final long serialVersionUID = 1L;

    public static final long RECOVERY_WINDOW_MS = 2L * 3600_000L;      // 2 hours
    public static final long GRACE_PERIOD_MS = 5L * 60_000L;           // 5 min grace
    public static final long MAX_ACTIVITY_DURATION_MS = 4L * 3600_000L; // 4h cap
    public static final long MAX_WINDOW_MS = MAX_ACTIVITY_DURATION_MS + RECOVERY_WINDOW_MS + GRACE_PERIOD
    public static final long PRE_EXERCISE_BP_LOOKBACK_MS = 30L * 60_000L; // 30 min
    public static final long PRE_EXERCISE_HR_LOOKBACK_MS = 10L * 60_000L; // 10 min
    public static final long CONCURRENT_ACTIVITY_THRESHOLD_MS = 30L * 60_000L; // 30 min

    @JsonProperty("patientId")
    private String patientId;

    @JsonProperty("patientAge")
    private Integer patientAge;

    @JsonProperty("hrMax")
    private double hrMax;

    @JsonProperty("activeSessions")
    private Map<String, ActivitySession> activeSessions;

    @JsonProperty("lastRestingHR")
    private Double lastRestingHR;

    @JsonProperty("lastRestingHRTimestamp")
    private Long lastRestingHRTimestamp;

    @JsonProperty("lastBPSystolic")
    private Double lastBPSystolic;

    @JsonProperty("lastBPDiastolic")
```

```
    private Double lastBPDiastolic;

    @JsonProperty("lastBPTimestamp")
    private Long lastBPTimestamp;

    @JsonProperty("totalActivitiesProcessed")
    private long totalActivitiesProcessed;

    @JsonProperty("lastUpdated")
    private long lastUpdated;

    public ActivityCorrelationState() {
        this.activeSessions = new HashMap<>();
        this.hrMax = ActivityIntensityZone.DEFAULT_HR_MAX;
    }

    public ActivityCorrelationState(String patientId) {
        this();
        this.patientId = patientId;
    }

    /**
     * Set patient age and recompute HR_max using Tanaka formula.
     */
    public void setPatientAge(Integer age) {
        this.patientAge = age;
        if (age != null && age > 0) {
            this.hrMax = ActivityIntensityZone.estimateHRMax(age);
        }
    }

    /**
     * Open a new activity session. Returns the timer fire time.
     *
     * @param activityEventId unique event ID
     * @param activityStart   timestamp of activity start
     * @param durationMs      reported activity duration in ms (0 if unknown, capped at MAX_ACTIVITY_DURA
     * @param payload         activity event payload (exercise_type, mets, etc.)
     * @return timer fire time in epoch ms
     */
    public long openSession(String activityEventId, long activityStart,
                            long durationMs, Map<String, Object> payload) {
        ActivitySession session = new ActivitySession();
        session.activityEventId = activityEventId;
        session.activityStartTime = activityStart;
        session.activityPayload = payload != null ? new HashMap<>(payload) : new HashMap<>();

        // Cap duration
        long cappedDuration = Math.min(Math.max(durationMs, 30L * 60_000L), MAX_ACTIVITY_DURATION_MS);
        session.reportedDurationMs = durationMs;
        session.activityEndTime = activityStart + cappedDuration;

        // Setup HR window
        session.hrWindow = new HRWindow();
```

```
        session.hrWindow.setActivityStartTime(activityStart);
        session.hrWindow.setActivityEndTime(session.activityEndTime);
        session.hrWindow.setHrMax(hrMax);
        if (lastRestingHR != null) {
            session.hrWindow.setRestingHR(lastRestingHR);
        }

        // Setup glucose window (reuse Module 10's GlucoseWindow for glucose readings)
        session.glucoseWindow = new GlucoseWindow();
        session.glucoseWindow.setWindowOpenTime(activityStart);
        session.glucoseWindow.setWindowCloseTime(session.activityEndTime + RECOVERY_WINDOW_MS);

        // Setup BP window
        session.bpWindow = new BPWindow();

        // Retroactive pre-exercise BP: attach if within 30 min
        if (lastBPTimestamp != null
                && (activityStart - lastBPTimestamp) <= PRE_EXERCISE_BP_LOOKBACK_MS
                && lastBPSystolic != null) {
            session.bpWindow.setPreMealSBP(lastBPSystolic);
            session.bpWindow.setPreMealDBP(lastBPDiastolic);
            session.bpWindow.setPreMealTimestamp(lastBPTimestamp);
        }

        // Parse exercise type from payload
        Object typeObj = session.activityPayload.get("exercise_type");
        session.exerciseType = ExerciseType.fromString(typeObj != null ? typeObj.toString() : null);

        // Parse METs from payload
        Object metsObj = session.activityPayload.get("mets");
        if (metsObj instanceof Number) {
            session.reportedMETs = ((Number) metsObj).doubleValue();
        } else {
            session.reportedMETs = session.exerciseType.getDefaultMETs();
        }

        // Check concurrent activities
        for (ActivitySession existing : activeSessions.values()) {
            if (Math.abs(activityStart - existing.activityStartTime) < CONCURRENT_ACTIVITY_THRESHOLD_MS)
                session.concurrent = true;
                existing.concurrent = true;
            }
        }

        activeSessions.put(activityEventId, session);
        totalActivitiesProcessed++;

        // Timer: activity end + 2h recovery + 5 min grace
        long timerFireTime = session.activityEndTime + RECOVERY_WINDOW_MS + GRACE_PERIOD_MS;
        session.timerFireTime = timerFireTime;
        return timerFireTime;
    }

    /**
```

```
     * Add an HR reading to all active sessions whose window hasn't closed.
     */
    public void addHRReading(long timestamp, double heartRate, String source) {
        for (ActivitySession session : activeSessions.values()) {
            long windowEnd = session.activityEndTime + RECOVERY_WINDOW_MS;
            // Accept HR readings from 10 min before activity start through recovery
            if (timestamp >= (session.activityStartTime - PRE_EXERCISE_HR_LOOKBACK_MS)
                    && timestamp <= windowEnd) {
                session.hrWindow.addReading(timestamp, heartRate, source);
            }
        }
    }

    /**
     * Add a glucose reading to all active sessions.
     */
    public void addGlucoseReading(long timestamp, double value, String source) {
        for (ActivitySession session : activeSessions.values()) {
            long windowEnd = session.activityEndTime + RECOVERY_WINDOW_MS;
            // Accept glucose from 30 min before activity through recovery
            long lookback = 30L * 60_000L;
            if (timestamp >= (session.activityStartTime - lookback) && timestamp <= windowEnd) {
                session.glucoseWindow.addReading(timestamp, value, source);
                // Set baseline from pre-exercise reading
                if (session.glucoseWindow.getBaseline() == null
                        && timestamp < session.activityStartTime) {
                    session.glucoseWindow.setBaseline(value);
                }
            }
        }
    }

    /**
     * Add a BP reading: buffer as lastBP AND feed to active sessions.
     * During exercise: captures peak exercise BP.
     * Post exercise: captures recovery BP.
     */
    public void addBPReading(long timestamp, double sbp, double dbp) {
        this.lastBPSystolic = sbp;
        this.lastBPDiastolic = dbp;
        this.lastBPTimestamp = timestamp;

        for (ActivitySession session : activeSessions.values()) {
            // During exercise: track peak BP
            if (timestamp >= session.activityStartTime && timestamp <= session.activityEndTime) {
                if (session.peakExerciseSBP == null || sbp > session.peakExerciseSBP) {
                    session.peakExerciseSBP = sbp;
                    session.peakExerciseDBP = dbp;
                    session.peakExerciseBPTimestamp = timestamp;
                }
            }
            // Post exercise: capture first recovery BP (5-15 min after activity end)
            if (session.activityEndTime != null
                    && !session.bpWindow.hasPostMeal()
```

```
                    && timestamp > session.activityEndTime
                    && timestamp <= session.activityEndTime + RECOVERY_WINDOW_MS) {
                session.bpWindow.setPostMealSBP(sbp);
                session.bpWindow.setPostMealDBP(dbp);
                session.bpWindow.setPostMealTimestamp(timestamp);
            }
        }
    }

    /**
     * Update resting HR from a resting-state reading.
     */
    public void updateRestingHR(double restingHR, long timestamp) {
        this.lastRestingHR = restingHR;
        this.lastRestingHRTimestamp = timestamp;
    }

    public ActivitySession closeSession(String activityEventId) {
        return activeSessions.remove(activityEventId);
    }

    public List<String> getSessionsForTimer(long timerTimestamp) {
        List<String> ids = new ArrayList<>();
        for (Map.Entry<String, ActivitySession> entry : activeSessions.entrySet()) {
            if (entry.getValue().timerFireTime == timerTimestamp) {
                ids.add(entry.getKey());
            }
        }
        return ids;
    }

    // --- Getters/Setters ---
    public String getPatientId() { return patientId; }
    public void setPatientId(String id) { this.patientId = id; }
    public Integer getPatientAge() { return patientAge; }
    public double getHrMax() { return hrMax; }
    public void setHrMax(double v) { this.hrMax = v; }
    public Map<String, ActivitySession> getActiveSessions() { return activeSessions; }
    public long getTotalActivitiesProcessed() { return totalActivitiesProcessed; }
    public void setTotalActivitiesProcessed(long c) { this.totalActivitiesProcessed = c; }
    public long getLastUpdated() { return lastUpdated; }
    public void setLastUpdated(long ts) { this.lastUpdated = ts; }
    public Double getLastRestingHR() { return lastRestingHR; }
    public Double getLastBPSystolic() { return lastBPSystolic; }
    public Double getLastBPDiastolic() { return lastBPDiastolic; }
    public Long getLastBPTimestamp() { return lastBPTimestamp; }

    /**
     * Per-activity session: tracks HR window, glucose window, BP, and exercise metadata.
     */
    @JsonIgnoreProperties(ignoreUnknown = true)
    public static class ActivitySession implements Serializable {
        private static final long serialVersionUID = 1L;
```

```
        @JsonProperty("activityEventId")
        public String activityEventId;

        @JsonProperty("activityStartTime")
        public long activityStartTime;

        @JsonProperty("activityEndTime")
        public Long activityEndTime;

        @JsonProperty("reportedDurationMs")
        public long reportedDurationMs;

        @JsonProperty("activityPayload")
        public Map<String, Object> activityPayload;

        @JsonProperty("exerciseType")
        public ExerciseType exerciseType;

        @JsonProperty("reportedMETs")
        public double reportedMETs;

        @JsonProperty("hrWindow")
        public HRWindow hrWindow;

        @JsonProperty("glucoseWindow")
        public GlucoseWindow glucoseWindow;

        @JsonProperty("bpWindow")
        public BPWindow bpWindow;

        @JsonProperty("peakExerciseSBP")
        public Double peakExerciseSBP;

        @JsonProperty("peakExerciseDBP")
        public Double peakExerciseDBP;

        @JsonProperty("peakExerciseBPTimestamp")
        public Long peakExerciseBPTimestamp;

        @JsonProperty("timerFireTime")
        public long timerFireTime;

        @JsonProperty("concurrent")
        public boolean concurrent;

        public ActivitySession() {
            this.activityPayload = new HashMap<>();
            this.exerciseType = ExerciseType.MIXED;
        }
    }
}
```

- [ ] **Step 2: Compile**

Run: `cd backend/shared-infrastructure/flink-processing && mvn compile -pl . -q 2>&1 | tail -5`
Expected: BUILD SUCCESS

- [ ] **Step 3: Commit**

```
git add backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/ActivityC
git commit -m "feat(module11): add ActivityCorrelationState with exercise session window management"
```

Task 4: Create ActivityResponseRecord (32 fields)

Files: - Create: `models/ActivityResponseRecord.java`

- [ ] **Step 1: Create ActivityResponseRecord**

```
package com.cardiofit.flink.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;
import java.io.Serializable;
import java.util.Map;

/**
 * Output record from Module 11: per-activity HR, glucose, and BP response.
 * 32 fields across all data configurations. Patients without CGM will have null glucose fields.
 * Emitted to flink.activity-response topic.
 */
@JsonIgnoreProperties(ignoreUnknown = true)
public class ActivityResponseRecord implements Serializable {

    private static final long serialVersionUID = 1L;

    // --- Identity (4 fields) ---
    @JsonProperty("recordId")
    private String recordId;

    @JsonProperty("patientId")
    private String patientId;

    @JsonProperty("activityEventId")
    private String activityEventId;

    @JsonProperty("correlationId")
    private String correlationId;

    // --- Activity metadata (5 fields) ---
    @JsonProperty("activityStartTime")
    private long activityStartTime;

    @JsonProperty("activityDurationMin")
```

```
    private double activityDurationMin;

    @JsonProperty("exerciseType")
    private ExerciseType exerciseType;

    @JsonProperty("reportedMETs")
    private double reportedMETs;

    @JsonProperty("metMinutes")
    private double metMinutes;

    // --- HR features (8 fields) ---
    @JsonProperty("restingHR")
    private Double restingHR;

    @JsonProperty("peakHR")
    private Double peakHR;

    @JsonProperty("meanActiveHR")
    private Double meanActiveHR;

    @JsonProperty("hrr1")
    private Double hrr1;

    @JsonProperty("hrr2")
    private Double hrr2;

    @JsonProperty("hrRecoveryClass")
    private HRRecoveryClass hrRecoveryClass;

    @JsonProperty("dominantZone")
    private ActivityIntensityZone dominantZone;

    @JsonProperty("hrReadingCount")
    private int hrReadingCount;

    // --- Glucose features (5 fields) ---
    @JsonProperty("preExerciseGlucose")
    private Double preExerciseGlucose;

    @JsonProperty("exerciseGlucoseDelta")
    private Double exerciseGlucoseDelta;

    @JsonProperty("glucoseNadir")
    private Double glucoseNadir;

    @JsonProperty("hypoglycemiaFlag")
    private boolean hypoglycemiaFlag;

    @JsonProperty("reboundHyperglycemiaFlag")
    private boolean reboundHyperglycemiaFlag;

    // --- BP features (4 fields) ---
    @JsonProperty("preExerciseSBP")
```

```
    private Double preExerciseSBP;

    @JsonProperty("peakExerciseSBP")
    private Double peakExerciseSBP;

    @JsonProperty("postExerciseSBP")
    private Double postExerciseSBP;

    @JsonProperty("exerciseBPResponse")
    private ExerciseBPResponse exerciseBPResponse;

    // --- Derived cardiac metrics (1 field) ---
    @JsonProperty("peakRPP")
    private Double peakRPP;

    // --- Processing metadata (5 fields) ---
    @JsonProperty("windowDurationMs")
    private long windowDurationMs;

    @JsonProperty("concurrent")
    private boolean concurrent;

    @JsonProperty("processingTimestamp")
    private long processingTimestamp;

    @JsonProperty("qualityScore")
    private double qualityScore;

    @JsonProperty("version")
    private String version;

    public ActivityResponseRecord() {
        this.version = "1.0";
        this.processingTimestamp = System.currentTimeMillis();
    }

    public static Builder builder() { return new Builder(); }

    public static class Builder {
        private final ActivityResponseRecord r = new ActivityResponseRecord();

        public Builder recordId(String v) { r.recordId = v; return this; }
        public Builder patientId(String v) { r.patientId = v; return this; }
        public Builder activityEventId(String v) { r.activityEventId = v; return this; }
        public Builder correlationId(String v) { r.correlationId = v; return this; }
        public Builder activityStartTime(long v) { r.activityStartTime = v; return this; }
        public Builder activityDurationMin(double v) { r.activityDurationMin = v; return this; }
        public Builder exerciseType(ExerciseType v) { r.exerciseType = v; return this; }
        public Builder reportedMETs(double v) { r.reportedMETs = v; return this; }
        public Builder metMinutes(double v) { r.metMinutes = v; return this; }
        public Builder restingHR(Double v) { r.restingHR = v; return this; }
        public Builder peakHR(Double v) { r.peakHR = v; return this; }
        public Builder meanActiveHR(Double v) { r.meanActiveHR = v; return this; }
        public Builder hrr1(Double v) { r.hrr1 = v; return this; }
```

```
        public Builder hrr2(Double v) { r.hrr2 = v; return this; }
        public Builder hrRecoveryClass(HRRecoveryClass v) { r.hrRecoveryClass = v; return this; }
        public Builder dominantZone(ActivityIntensityZone v) { r.dominantZone = v; return this; }
        public Builder hrReadingCount(int v) { r.hrReadingCount = v; return this; }
        public Builder preExerciseGlucose(Double v) { r.preExerciseGlucose = v; return this; }
        public Builder exerciseGlucoseDelta(Double v) { r.exerciseGlucoseDelta = v; return this; }
        public Builder glucoseNadir(Double v) { r.glucoseNadir = v; return this; }
        public Builder hypoglycemiaFlag(boolean v) { r.hypoglycemiaFlag = v; return this; }
        public Builder reboundHyperglycemiaFlag(boolean v) { r.reboundHyperglycemiaFlag = v; return this;
        public Builder preExerciseSBP(Double v) { r.preExerciseSBP = v; return this; }
        public Builder peakExerciseSBP(Double v) { r.peakExerciseSBP = v; return this; }
        public Builder postExerciseSBP(Double v) { r.postExerciseSBP = v; return this; }
        public Builder exerciseBPResponse(ExerciseBPResponse v) { r.exerciseBPResponse = v; return this;
        public Builder peakRPP(Double v) { r.peakRPP = v; return this; }
        public Builder windowDurationMs(long v) { r.windowDurationMs = v; return this; }
        public Builder concurrent(boolean v) { r.concurrent = v; return this; }
        public Builder qualityScore(double v) { r.qualityScore = v; return this; }
        public ActivityResponseRecord build() { return r; }
    }

    // --- Getters ---
    public String getRecordId() { return recordId; }
    public String getPatientId() { return patientId; }
    public String getActivityEventId() { return activityEventId; }
    public String getCorrelationId() { return correlationId; }
    public long getActivityStartTime() { return activityStartTime; }
    public double getActivityDurationMin() { return activityDurationMin; }
    public ExerciseType getExerciseType() { return exerciseType; }
    public double getReportedMETs() { return reportedMETs; }
    public double getMetMinutes() { return metMinutes; }
    public Double getRestingHR() { return restingHR; }
    public Double getPeakHR() { return peakHR; }
    public Double getMeanActiveHR() { return meanActiveHR; }
    public Double getHrr1() { return hrr1; }
    public Double getHrr2() { return hrr2; }
    public HRRecoveryClass getHrRecoveryClass() { return hrRecoveryClass; }
    public ActivityIntensityZone getDominantZone() { return dominantZone; }
    public int getHrReadingCount() { return hrReadingCount; }
    public Double getPreExerciseGlucose() { return preExerciseGlucose; }
    public Double getExerciseGlucoseDelta() { return exerciseGlucoseDelta; }
    public Double getGlucoseNadir() { return glucoseNadir; }
    public boolean isHypoglycemiaFlag() { return hypoglycemiaFlag; }
    public boolean isReboundHyperglycemiaFlag() { return reboundHyperglycemiaFlag; }
    public Double getPreExerciseSBP() { return preExerciseSBP; }
    public Double getPeakExerciseSBP() { return peakExerciseSBP; }
    public Double getPostExerciseSBP() { return postExerciseSBP; }
    public ExerciseBPResponse getExerciseBPResponse() { return exerciseBPResponse; }
    public Double getPeakRPP() { return peakRPP; }
    public long getWindowDurationMs() { return windowDurationMs; }
    public boolean isConcurrent() { return concurrent; }
    public long getProcessingTimestamp() { return processingTimestamp; }
    public double getQualityScore() { return qualityScore; }
    public String getVersion() { return version; }
    public void setCorrelationId(String v) { this.correlationId = v; }
```

```
    public void setRecordId(String v) { this.recordId = v; }

    @Override
    public String toString() {
        return "ActivityResponseRecord{" +
                "patientId='" + patientId + '\'' +
                ", activityEventId='" + activityEventId + '\'' +
                ", type=" + exerciseType +
                ", peakHR=" + peakHR +
                ", hrr1=" + hrr1 +
                ", hrrClass=" + hrRecoveryClass +
                ", glucoseDelta=" + exerciseGlucoseDelta +
                ", bpResponse=" + exerciseBPResponse +
                '}';
    }
}
```

- [ ] **Step 2: Compile**

Run: `cd backend/shared-infrastructure/flink-processing && mvn compile -pl . -q 2>&1 | tail -5`
Expected: BUILD SUCCESS

- [ ] **Step 3: Commit**

```
git add backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/ActivityR
git commit -m "feat(module11): add ActivityResponseRecord output model (32 fields)"
```

Task 5: Create FitnessPatternSummary (24 fields) and FitnessPatternState

Files: - Create: `models/FitnessPatternSummary.java` - Create: `models/FitnessPatternState.java`

- [ ] **Step 1: Create FitnessPatternSummary**

```
package com.cardiofit.flink.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;
import java.io.Serializable;
import java.util.List;
import java.util.Map;

/**
 * Output record from Module 11b: weekly fitness pattern aggregation.
 * 24 fields. Emitted to flink.fitness-patterns topic.
 */
@JsonIgnoreProperties(ignoreUnknown = true)
public class FitnessPatternSummary implements Serializable {
```

```
    private static final long serialVersionUID = 1L;

    // --- Identity (3 fields) ---
    @JsonProperty("summaryId")
    private String summaryId;

    @JsonProperty("patientId")
    private String patientId;

    @JsonProperty("correlationId")
    private String correlationId;

    // --- Time range (2 fields) ---
    @JsonProperty("periodStartMs")
    private long periodStartMs;

    @JsonProperty("periodEndMs")
    private long periodEndMs;

    // --- Weekly exercise dose (3 fields) ---
    @JsonProperty("totalMetMinutes")
    private double totalMetMinutes;

    @JsonProperty("totalActiveDurationMin")
    private double totalActiveDurationMin;

    @JsonProperty("activityCount")
    private int activityCount;

    // --- Aggregated HR metrics (4 fields) ---
    @JsonProperty("meanPeakHR")
    private Double meanPeakHR;

    @JsonProperty("meanHRR1")
    private Double meanHRR1;

    @JsonProperty("dominantHRRecoveryClass")
    private HRRecoveryClass dominantHRRecoveryClass;

    @JsonProperty("zoneDistributionPct")
    private Map<ActivityIntensityZone, Double> zoneDistributionPct;

    // --- VO2max and fitness (3 fields) ---
    @JsonProperty("estimatedVO2max")
    private Double estimatedVO2max;

    @JsonProperty("fitnessLevel")
    private FitnessLevel fitnessLevel;

    @JsonProperty("vo2maxTrend")
    private Double vo2maxTrend;

    // --- Glucose-exercise response (3 fields) ---
    @JsonProperty("meanExerciseGlucoseDelta")
```

```java
    private Double meanExerciseGlucoseDelta;

    @JsonProperty("hypoglycemiaEventCount")
    private int hypoglycemiaEventCount;

    @JsonProperty("meanGlucoseDropAerobic")
    private Double meanGlucoseDropAerobic;

    // --- Exercise type breakdown (1 field — map) ---
    @JsonProperty("exerciseTypeBreakdown")
    private Map<ExerciseType, ExerciseTypeStats> exerciseTypeBreakdown;

    // --- Processing metadata (5 fields) ---
    @JsonProperty("processingTimestamp")
    private long processingTimestamp;

    @JsonProperty("qualityScore")
    private double qualityScore;

    @JsonProperty("sessionsWithHR")
    private int sessionsWithHR;

    @JsonProperty("sessionsWithGlucose")
    private int sessionsWithGlucose;

    @JsonProperty("version")
    private String version;

    public FitnessPatternSummary() {
        this.version = "1.0";
        this.processingTimestamp = System.currentTimeMillis();
    }

    // --- Getters/Setters ---
    public String getSummaryId() { return summaryId; }
    public void setSummaryId(String v) { this.summaryId = v; }
    public String getPatientId() { return patientId; }
    public void setPatientId(String v) { this.patientId = v; }
    public String getCorrelationId() { return correlationId; }
    public void setCorrelationId(String v) { this.correlationId = v; }
    public long getPeriodStartMs() { return periodStartMs; }
    public void setPeriodStartMs(long v) { this.periodStartMs = v; }
    public long getPeriodEndMs() { return periodEndMs; }
    public void setPeriodEndMs(long v) { this.periodEndMs = v; }
    public double getTotalMetMinutes() { return totalMetMinutes; }
    public void setTotalMetMinutes(double v) { this.totalMetMinutes = v; }
    public double getTotalActiveDurationMin() { return totalActiveDurationMin; }
    public void setTotalActiveDurationMin(double v) { this.totalActiveDurationMin = v; }
    public int getActivityCount() { return activityCount; }
    public void setActivityCount(int v) { this.activityCount = v; }
    public Double getMeanPeakHR() { return meanPeakHR; }
    public void setMeanPeakHR(Double v) { this.meanPeakHR = v; }
    public Double getMeanHRR1() { return meanHRR1; }
    public void setMeanHRR1(Double v) { this.meanHRR1 = v; }
```

```
    public HRRecoveryClass getDominantHRRecoveryClass() { return dominantHRRecoveryClass; }
    public void setDominantHRRecoveryClass(HRRecoveryClass v) { this.dominantHRRecoveryClass = v; }
    public Map<ActivityIntensityZone, Double> getZoneDistributionPct() { return zoneDistributionPct; }
    public void setZoneDistributionPct(Map<ActivityIntensityZone, Double> v) { this.zoneDistributionPct =
    public Double getEstimatedVO2max() { return estimatedVO2max; }
    public void setEstimatedVO2max(Double v) { this.estimatedVO2max = v; }
    public FitnessLevel getFitnessLevel() { return fitnessLevel; }
    public void setFitnessLevel(FitnessLevel v) { this.fitnessLevel = v; }
    public Double getVo2maxTrend() { return vo2maxTrend; }
    public void setVo2maxTrend(Double v) { this.vo2maxTrend = v; }
    public Double getMeanExerciseGlucoseDelta() { return meanExerciseGlucoseDelta; }
    public void setMeanExerciseGlucoseDelta(Double v) { this.meanExerciseGlucoseDelta = v; }
    public int getHypoglycemiaEventCount() { return hypoglycemiaEventCount; }
    public void setHypoglycemiaEventCount(int v) { this.hypoglycemiaEventCount = v; }
    public Double getMeanGlucoseDropAerobic() { return meanGlucoseDropAerobic; }
    public void setMeanGlucoseDropAerobic(Double v) { this.meanGlucoseDropAerobic = v; }
    public Map<ExerciseType, ExerciseTypeStats> getExerciseTypeBreakdown() { return exerciseTypeBreakdown
    public void setExerciseTypeBreakdown(Map<ExerciseType, ExerciseTypeStats> v) { this.exerciseTypeBreak
    public long getProcessingTimestamp() { return processingTimestamp; }
    public double getQualityScore() { return qualityScore; }
    public void setQualityScore(double v) { this.qualityScore = v; }
    public int getSessionsWithHR() { return sessionsWithHR; }
    public void setSessionsWithHR(int v) { this.sessionsWithHR = v; }
    public int getSessionsWithGlucose() { return sessionsWithGlucose; }
    public void setSessionsWithGlucose(int v) { this.sessionsWithGlucose = v; }
    public String getVersion() { return version; }

    @JsonIgnoreProperties(ignoreUnknown = true)
    public static class ExerciseTypeStats implements Serializable {
        private static final long serialVersionUID = 1L;

        @JsonProperty("sessionCount")
        public int sessionCount;

        @JsonProperty("totalMetMinutes")
        public double totalMetMinutes;

        @JsonProperty("meanPeakHR")
        public double meanPeakHR;

        @JsonProperty("meanGlucoseDelta")
        public Double meanGlucoseDelta;

        public ExerciseTypeStats() {}
    }
}
```

- [ ] **Step 2: Create FitnessPatternState**

```
package com.cardiofit.flink.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;
```

```java
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

/**
 * Per-patient state for Module 11b weekly aggregation.
 *
 * Stores:
 * - 90-day rolling buffer of VO2max estimates for fitness trend analysis
 * - Weekly accumulator of ActivityResponseRecords
 * - Running resting HR baseline for VO2max estimation
 *
 * State TTL: 90 days (VO2max trend window).
 * Weekly timer fires every Monday 00:00 UTC.
 */
@JsonIgnoreProperties(ignoreUnknown = true)
public class FitnessPatternState implements Serializable {

    private static final long serialVersionUID = 1L;

    public static final int VO2MAX_BUFFER_MAX_DAYS = 90;
    public static final long WEEK_MS = 7L * 86_400_000L;

    @JsonProperty("patientId")
    private String patientId;

    @JsonProperty("patientAge")
    private Integer patientAge;

    @JsonProperty("hrMax")
    private double hrMax;

    @JsonProperty("lastKnownRestingHR")
    private Double lastKnownRestingHR;

    // --- VO2max trend buffer (90-day rolling) ---
    @JsonProperty("vo2maxEstimates")
    private List<VO2maxEstimate> vo2maxEstimates;

    // --- Weekly accumulators ---
    @JsonProperty("weeklyActivityRecords")
    private List<ActivityResponseRecord> weeklyActivityRecords;

    // --- Timer state ---
    @JsonProperty("weeklyTimerRegistered")
    private boolean weeklyTimerRegistered;

    @JsonProperty("lastWeeklyEmitTimestamp")
    private long lastWeeklyEmitTimestamp;

    @JsonProperty("lastUpdated")
    private long lastUpdated;

    @JsonProperty("totalRecordsProcessed")
```

```java
    private long totalRecordsProcessed;

    public FitnessPatternState() {
        this.vo2maxEstimates = new ArrayList<>();
        this.weeklyActivityRecords = new ArrayList<>();
        this.hrMax = ActivityIntensityZone.DEFAULT_HR_MAX;
    }

    public FitnessPatternState(String patientId) {
        this();
        this.patientId = patientId;
    }

    public void addActivityRecord(ActivityResponseRecord record) {
        weeklyActivityRecords.add(record);
        totalRecordsProcessed++;
    }

    public void addVO2maxEstimate(double vo2max, long timestamp) {
        vo2maxEstimates.add(new VO2maxEstimate(vo2max, timestamp));
        // Evict estimates older than 90 days
        long cutoff = timestamp - (VO2MAX_BUFFER_MAX_DAYS * 86_400_000L);
        vo2maxEstimates.removeIf(e -> e.timestamp < cutoff);
    }

    public List<ActivityResponseRecord> drainWeeklyRecords() {
        List<ActivityResponseRecord> drained = new ArrayList<>(weeklyActivityRecords);
        weeklyActivityRecords.clear();
        return drained;
    }

    /**
     * Compute VO2max trend as slope of linear regression on (timestamp, vo2max) pairs.
     * Returns mL/kg/min per week. Positive = improving.
     */
    public Double computeVO2maxTrendPerWeek() {
        if (vo2maxEstimates.size() < 3) return null;
        int n = vo2maxEstimates.size();
        double sumX = 0, sumY = 0;
        for (VO2maxEstimate e : vo2maxEstimates) {
            sumX += e.timestamp;
            sumY += e.vo2max;
        }
        double meanX = sumX / n;
        double meanY = sumY / n;
        double ssXX = 0, ssXY = 0;
        for (VO2maxEstimate e : vo2maxEstimates) {
            double dx = e.timestamp - meanX;
            ssXX += dx * dx;
            ssXY += dx * (e.vo2max - meanY);
        }
        if (ssXX < 1e-12) return 0.0;
        double slopePerMs = ssXY / ssXX;
        return slopePerMs * WEEK_MS; // convert to per-week
```

```
    }

    // --- Getters/Setters ---
    public String getPatientId() { return patientId; }
    public void setPatientId(String v) { this.patientId = v; }
    public Integer getPatientAge() { return patientAge; }
    public void setPatientAge(Integer v) { this.patientAge = v; }
    public double getHrMax() { return hrMax; }
    public void setHrMax(double v) { this.hrMax = v; }
    public Double getLastKnownRestingHR() { return lastKnownRestingHR; }
    public void setLastKnownRestingHR(Double v) { this.lastKnownRestingHR = v; }
    public List<VO2maxEstimate> getVo2maxEstimates() { return vo2maxEstimates; }
    public List<ActivityResponseRecord> getWeeklyActivityRecords() { return weeklyActivityRecords; }
    public boolean isWeeklyTimerRegistered() { return weeklyTimerRegistered; }
    public void setWeeklyTimerRegistered(boolean v) { this.weeklyTimerRegistered = v; }
    public long getLastWeeklyEmitTimestamp() { return lastWeeklyEmitTimestamp; }
    public void setLastWeeklyEmitTimestamp(long v) { this.lastWeeklyEmitTimestamp = v; }
    public long getLastUpdated() { return lastUpdated; }
    public void setLastUpdated(long v) { this.lastUpdated = v; }
    public long getTotalRecordsProcessed() { return totalRecordsProcessed; }

    @JsonIgnoreProperties(ignoreUnknown = true)
    public static class VO2maxEstimate implements Serializable {
        private static final long serialVersionUID = 1L;

        @JsonProperty("vo2max")
        public double vo2max;

        @JsonProperty("timestamp")
        public long timestamp;

        public VO2maxEstimate() {}

        public VO2maxEstimate(double vo2max, long timestamp) {
            this.vo2max = vo2max;
            this.timestamp = timestamp;
        }
    }
}
```

- [ ] **Step 3: Compile**

Run: `cd backend/shared-infrastructure/flink-processing && mvn compile -pl . -q 2>&1 | tail -5`
Expected: BUILD SUCCESS

- [ ] **Step 4: Commit**

```
git add \
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/FitnessPatternS
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/FitnessPatternS
git commit -m "feat(module11b): add FitnessPatternSummary (24 fields) and FitnessPatternState with 90-day
```

Task 6: Create Module11HRAnalyzer

Files: - Create: `operators/Module11HRAnalyzer.java` - Test: `operators/Module11HRAnalyzerTest.java`

- [ ] **Step 1: Write the failing test**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class Module11HRAnalyzerTest {

    private static final long BASE = 1743552000000L; // 2025-04-02 00:00 UTC
    private static final long MIN_1 = 60_000L;
    private static final long MIN_5 = 5 * 60_000L;

    @Test
    void peakHR_correctlyIdentified() {
        HRWindow window = exerciseWindow(
                70, 85, 100, 120, 140, 155, 165, 170, 168, 160, 140, 120, 100, 85
        );
        Module11HRAnalyzer.Result result = Module11HRAnalyzer.analyze(window);
        assertEquals(170.0, result.peakHR);
    }

    @Test
    void meanActiveHR_excludesPreAndRecovery() {
        // Pre-exercise (before activityStartTime): 70
        // Active: 120, 140, 160, 150 → mean = 142.5
        // Recovery (after activityEndTime): 100, 85
        HRWindow window = new HRWindow();
        long actStart = BASE;
        long actEnd = BASE + 20 * MIN_1;
        window.setActivityStartTime(actStart);
        window.setActivityEndTime(actEnd);
        window.setHrMax(180.0);
        window.setRestingHR(65.0);

        window.addReading(BASE - 5 * MIN_1, 70, "WEARABLE");  // pre
        window.addReading(BASE, 120, "WEARABLE");               // active
        window.addReading(BASE + 5 * MIN_1, 140, "WEARABLE");
        window.addReading(BASE + 10 * MIN_1, 160, "WEARABLE");
        window.addReading(BASE + 15 * MIN_1, 150, "WEARABLE");
        window.addReading(BASE + 25 * MIN_1, 100, "WEARABLE");  // recovery
        window.addReading(BASE + 30 * MIN_1, 85, "WEARABLE");
        window.sortByTime();

        Module11HRAnalyzer.Result result = Module11HRAnalyzer.analyze(window);
        assertEquals(142.5, result.meanActiveHR, 0.1);
```

```
    }

    @Test
    void hrr1_dropFromPeakAt1Minute() {
        HRWindow window = new HRWindow();
        long actStart = BASE;
        long actEnd = BASE + 30 * MIN_1;
        window.setActivityStartTime(actStart);
        window.setActivityEndTime(actEnd);
        window.setHrMax(180.0);

        // Active phase readings
        window.addReading(BASE + 25 * MIN_1, 165, "WEARABLE");
        window.addReading(BASE + 28 * MIN_1, 172, "WEARABLE"); // peak
        window.addReading(BASE + 30 * MIN_1, 170, "WEARABLE");
        // Recovery phase
        window.addReading(actEnd + 1 * MIN_1, 148, "WEARABLE"); // ~1 min post: HRR1 = 172-148 = 24
        window.addReading(actEnd + 2 * MIN_1, 130, "WEARABLE"); // ~2 min post: HRR2 = 172-130 = 42
        window.sortByTime();

        Module11HRAnalyzer.Result result = Module11HRAnalyzer.analyze(window);
        assertEquals(172.0, result.peakHR);
        assertEquals(24.0, result.hrr1, 1.0);
        assertEquals(42.0, result.hrr2, 1.0);
        assertEquals(HRRecoveryClass.NORMAL, result.hrRecoveryClass);
    }

    @Test
    void hrr1_abnormalClassification() {
        HRWindow window = new HRWindow();
        long actStart = BASE;
        long actEnd = BASE + 30 * MIN_1;
        window.setActivityStartTime(actStart);
        window.setActivityEndTime(actEnd);
        window.setHrMax(180.0);

        window.addReading(BASE + 28 * MIN_1, 160, "WEARABLE"); // peak
        window.addReading(actEnd + 1 * MIN_1, 152, "WEARABLE"); // HRR1 = 8 → ABNORMAL
        window.addReading(actEnd + 2 * MIN_1, 145, "WEARABLE");
        window.sortByTime();

        Module11HRAnalyzer.Result result = Module11HRAnalyzer.analyze(window);
        assertEquals(8.0, result.hrr1, 1.0);
        assertEquals(HRRecoveryClass.ABNORMAL, result.hrRecoveryClass);
        assertTrue(result.hrRecoveryClass.isPrognosticFlag());
    }

    @Test
    void dominantZone_correctlyComputed() {
        HRWindow window = exerciseWindow(
                // All readings in ZONE_3_TEMPO for hrMax=180: 70-79% = 126-142
                130, 135, 138, 140, 138, 135, 132, 130
        );
        Module11HRAnalyzer.Result result = Module11HRAnalyzer.analyze(window);
```

```
        assertEquals(ActivityIntensityZone.ZONE_3_TEMPO, result.dominantZone);
    }

    @Test
    void rpp_computedFromPeakHRAndPeakSBP() {
        HRWindow window = new HRWindow();
        long actStart = BASE;
        long actEnd = BASE + 30 * MIN_1;
        window.setActivityStartTime(actStart);
        window.setActivityEndTime(actEnd);
        window.setHrMax(180.0);
        window.addReading(BASE + 15 * MIN_1, 160, "WEARABLE");
        window.sortByTime();

        Module11HRAnalyzer.Result result = Module11HRAnalyzer.analyze(window);
        // RPP requires peak SBP to be set externally, so test that peakHR is correct
        assertEquals(160.0, result.peakHR);
    }

    @Test
    void emptyWindow_returnsNull() {
        HRWindow window = new HRWindow();
        window.setActivityStartTime(BASE);
        window.setActivityEndTime(BASE + 30 * MIN_1);
        Module11HRAnalyzer.Result result = Module11HRAnalyzer.analyze(window);
        assertNull(result);
    }

    /** Helper: create exercise window with 5-min interval readings starting at BASE */
    private HRWindow exerciseWindow(double... values) {
        HRWindow w = new HRWindow();
        long actStart = BASE;
        long actEnd = BASE + (values.length - 1) * MIN_5;
        w.setActivityStartTime(actStart);
        w.setActivityEndTime(actEnd);
        w.setHrMax(180.0);
        w.setRestingHR(65.0);
        for (int i = 0; i < values.length; i++) {
            w.addReading(BASE + i * MIN_5, values[i], "WEARABLE");
        }
        w.sortByTime();
        return w;
    }
}
```

- [ ] **Step 2: Run test to verify it fails**

Run: `cd backend/shared-infrastructure/flink-processing && mvn test -pl . -Dtest=Module11HRAnalyzerTest -q 2>&1 | tail -10` Expected: FAIL — Module11HRAnalyzer class does not exist

- [ ] **Step 3: Write Module11HRAnalyzer implementation**

```java
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.*;
import java.util.List;
import java.util.Map;

/**
 * Heart rate feature extraction for Module 11.
 *
 * Computes from an HRWindow:
 * 1. peakHR — maximum HR during active exercise phase
 * 2. meanActiveHR — mean HR during active phase only (excludes pre/recovery)
 * 3. hrr1 — HR drop from peak at ~1 min post-exercise (HRR₁)
 * 4. hrr2 — HR drop from peak at ~2 min post-exercise (HRR₂)
 * 5. hrRecoveryClass — classification from HRR₁ (Cole et al., NEJM 1999)
 * 6. dominantZone — zone with most time during active phase
 * 7. readingCount — total HR readings in window
 * 8. qualityScore — 0-1 based on reading density and phase coverage
 *
 * HRR algorithm:
 * - Find peak HR during active phase
 * - In recovery readings, find reading closest to 60s post-activity-end → HRR₁
 * - Find reading closest to 120s post-activity-end → HRR₂
 * - If no recovery reading within 30s of target time, mark as null
 *
 * Stateless utility class.
 */
public class Module11HRAnalyzer {

    private static final long HRR1_TARGET_MS = 60_000L;  // 1 minute
    private static final long HRR2_TARGET_MS = 120_000L; // 2 minutes
    private static final long HRR_TOLERANCE_MS = 30_000L; // ±30 seconds

    private Module11HRAnalyzer() {}

    public static Result analyze(HRWindow window) {
        if (window == null || window.isEmpty()) return null;

        window.sortByTime();

        List<HRWindow.HRReading> activeReadings = window.getActivePhaseReadings();
        List<HRWindow.HRReading> recoveryReadings = window.getRecoveryPhaseReadings();

        if (activeReadings.isEmpty()) return null;

        // Peak HR during active phase
        double peakHR = Double.NEGATIVE_INFINITY;
        for (HRWindow.HRReading r : activeReadings) {
            if (r.heartRate > peakHR) {
                peakHR = r.heartRate;
            }
        }
```

```
        // Mean active HR
        double sumHR = 0;
        for (HRWindow.HRReading r : activeReadings) {
            sumHR += r.heartRate;
        }
        double meanActiveHR = sumHR / activeReadings.size();

        // HRR₁ and HRR₂ from recovery phase
        Double hrr1 = null;
        Double hrr2 = null;
        HRRecoveryClass hrRecoveryClass = HRRecoveryClass.INSUFFICIENT_DATA;

        if (window.getActivityEndTime() != null && recoveryReadings.size() >= HRRecoveryClass.MIN_RECOVER
            long actEnd = window.getActivityEndTime();

            // Find reading closest to 1 min post-exercise
            HRWindow.HRReading closest1Min = findClosestReading(recoveryReadings, actEnd + HRR1_TARGET_MS
            if (closest1Min != null
                    && Math.abs((closest1Min.timestamp - actEnd) - HRR1_TARGET_MS) <= HRR_TOLERANCE_MS) {
                hrr1 = peakHR - closest1Min.heartRate;
                hrRecoveryClass = HRRecoveryClass.fromHRR1(hrr1);
            }

            // Find reading closest to 2 min post-exercise
            HRWindow.HRReading closest2Min = findClosestReading(recoveryReadings, actEnd + HRR2_TARGET_MS
            if (closest2Min != null
                    && Math.abs((closest2Min.timestamp - actEnd) - HRR2_TARGET_MS) <= HRR_TOLERANCE_MS) {
                hrr2 = peakHR - closest2Min.heartRate;
            }
        }

        // Dominant zone
        Map<ActivityIntensityZone, Long> zoneDistribution = window.computeZoneDistribution();
        ActivityIntensityZone dominantZone = ActivityIntensityZone.ZONE_1_RECOVERY;
        long maxTime = 0;
        for (Map.Entry<ActivityIntensityZone, Long> entry : zoneDistribution.entrySet()) {
            if (entry.getValue() > maxTime) {
                maxTime = entry.getValue();
                dominantZone = entry.getKey();
            }
        }

        // Quality score
        long activeDurationMs = window.getActivityEndTime() != null
                ? window.getActivityEndTime() - window.getActivityStartTime()
                : 0;
        double expectedReadings = Math.max(1, activeDurationMs / 60_000.0); // ~1 reading/min expected
        double hrQuality = Math.min(1.0, activeReadings.size() / expectedReadings);
        double recoveryQuality = recoveryReadings.size() >= 2 ? 0.3 : 0.0;
        double qualityScore = Math.min(1.0, hrQuality * 0.7 + recoveryQuality);

        Result r = new Result();
        r.peakHR = peakHR;
        r.meanActiveHR = meanActiveHR;
```

```
        r.hrr1 = hrr1;
        r.hrr2 = hrr2;
        r.hrRecoveryClass = hrRecoveryClass;
        r.dominantZone = dominantZone;
        r.zoneDistribution = zoneDistribution;
        r.readingCount = window.size();
        r.qualityScore = qualityScore;
        return r;
    }

    /**
     * Compute Rate-Pressure Product from peak HR and peak SBP.
     * RPP = HR × SBP. Normal resting ~7,000; peak exercise ~25,000–40,000.
     */
    public static Double computeRPP(Double peakHR, Double peakSBP) {
        if (peakHR == null || peakSBP == null) return null;
        return peakHR * peakSBP;
    }

    private static HRWindow.HRReading findClosestReading(List<HRWindow.HRReading> readings, long targetTi
        HRWindow.HRReading closest = null;
        long minDiff = Long.MAX_VALUE;
        for (HRWindow.HRReading r : readings) {
            long diff = Math.abs(r.timestamp - targetTime);
            if (diff < minDiff) {
                minDiff = diff;
                closest = r;
            }
        }
        return closest;
    }

    public static class Result {
        public double peakHR;
        public double meanActiveHR;
        public Double hrr1;
        public Double hrr2;
        public HRRecoveryClass hrRecoveryClass;
        public ActivityIntensityZone dominantZone;
        public Map<ActivityIntensityZone, Long> zoneDistribution;
        public int readingCount;
        public double qualityScore;
    }
}
```

- [ ] **Step 4: Run test to verify it passes**

Run: `cd backend/shared-infrastructure/flink-processing && mvn test -pl . -Dtest=Module11HRAnalyzerTest -q 2>&1 | tail -10` Expected: PASS (6 tests)

- [ ] **Step 5: Commit**

```
git add \
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/operators/Module11HRAn
  backend/shared-infrastructure/flink-processing/src/test/java/com/cardiofit/flink/operators/Module11HRAn
git commit -m "feat(module11): add Module11HRAnalyzer with HRR classification and zone distribution — 6 t
```

Task 7: Create Module11GlucoseExerciseAnalyzer

Files: - Create: `operators/Module11GlucoseExerciseAnalyzer.java` - Test: `operators/Module11GlucoseExerciseAnalyzerTest.java`

- [ ] **Step 1: Write the failing test**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class Module11GlucoseExerciseAnalyzerTest {

    private static final long BASE = 1743552000000L;
    private static final long MIN_5 = 5 * 60_000L;
    private static final long MIN_30 = 30 * 60_000L;

    @Test
    void aerobicExercise_glucoseDropsFromBaseline() {
        // Simulate 30 min moderate jog: glucose drops from 120 to 90
        GlucoseWindow window = new GlucoseWindow();
        window.setBaseline(120.0);
        long actStart = BASE;
        long actEnd = BASE + MIN_30;
        window.addReading(BASE - MIN_5, 120.0, "CGM");   // pre-exercise
        window.addReading(BASE + 10 * 60_000L, 110.0, "CGM");
        window.addReading(BASE + 20 * 60_000L, 100.0, "CGM");
        window.addReading(BASE + 30 * 60_000L, 90.0, "CGM");  // end of exercise
        window.addReading(BASE + 45 * 60_000L, 95.0, "CGM");  // recovery
        window.addReading(BASE + 60 * 60_000L, 105.0, "CGM");
        window.sortByTime();

        Module11GlucoseExerciseAnalyzer.Result result =
                Module11GlucoseExerciseAnalyzer.analyze(window, actStart, actEnd, ExerciseType.AEROBIC);

        assertNotNull(result);
        assertEquals(120.0, result.preExerciseGlucose);
        assertTrue(result.exerciseGlucoseDelta < 0); // glucose dropped
        assertEquals(90.0, result.glucoseNadir);
        assertFalse(result.hypoglycemiaFlag); // 90 > 70
    }
```

```
    @Test
    void hypoglycemiaFlagged_below70() {
        GlucoseWindow window = new GlucoseWindow();
        window.setBaseline(100.0);
        long actStart = BASE;
        long actEnd = BASE + MIN_30;
        window.addReading(BASE - MIN_5, 100.0, "CGM");
        window.addReading(BASE + 15 * 60_000L, 80.0, "CGM");
        window.addReading(BASE + 25 * 60_000L, 65.0, "CGM"); // hypo!
        window.addReading(BASE + 40 * 60_000L, 75.0, "CGM");
        window.sortByTime();

        Module11GlucoseExerciseAnalyzer.Result result =
                Module11GlucoseExerciseAnalyzer.analyze(window, actStart, actEnd, ExerciseType.AEROBIC);

        assertTrue(result.hypoglycemiaFlag);
        assertEquals(65.0, result.glucoseNadir);
    }

    @Test
    void hiitExercise_reboundHyperglycemiaDetected() {
        // HIIT: catecholamine spike, glucose rises above baseline + 40
        GlucoseWindow window = new GlucoseWindow();
        window.setBaseline(110.0);
        long actStart = BASE;
        long actEnd = BASE + 20 * 60_000L;
        window.addReading(BASE - MIN_5, 110.0, "CGM");
        window.addReading(BASE + 10 * 60_000L, 130.0, "CGM");
        window.addReading(BASE + 20 * 60_000L, 145.0, "CGM");
        window.addReading(BASE + 35 * 60_000L, 160.0, "CGM"); // rebound: 160 > 110+40=150
        window.addReading(BASE + 50 * 60_000L, 125.0, "CGM");
        window.sortByTime();

        Module11GlucoseExerciseAnalyzer.Result result =
                Module11GlucoseExerciseAnalyzer.analyze(window, actStart, actEnd, ExerciseType.HIIT);

        assertTrue(result.reboundHyperglycemiaFlag);
        assertTrue(result.exerciseGlucoseDelta > 0); // glucose went up during HIIT
    }

    @Test
    void noGlucoseReadings_returnsNull() {
        GlucoseWindow window = new GlucoseWindow();
        Module11GlucoseExerciseAnalyzer.Result result =
                Module11GlucoseExerciseAnalyzer.analyze(window, BASE, BASE + MIN_30, ExerciseType.AEROBIC
        assertNull(result);
    }

    @Test
    void preExerciseGlucose_fromReadingBeforeActivityStart() {
        GlucoseWindow window = new GlucoseWindow();
        window.setBaseline(115.0);
        long actStart = BASE;
        long actEnd = BASE + MIN_30;
```

```
        // Two pre-exercise readings — should pick the one closest to start
        window.addReading(BASE - 20 * 60_000L, 108.0, "CGM");
        window.addReading(BASE - 5 * 60_000L, 115.0, "CGM"); // closest
        window.addReading(BASE + 15 * 60_000L, 100.0, "CGM");
        window.sortByTime();

        Module11GlucoseExerciseAnalyzer.Result result =
                Module11GlucoseExerciseAnalyzer.analyze(window, actStart, actEnd, ExerciseType.AEROBIC);

        assertEquals(115.0, result.preExerciseGlucose, 0.1);
    }
}
```

- [ ] **Step 2: Run test to verify it fails**

Run: `cd backend/shared-infrastructure/flink-processing && mvn test -pl . -Dtest=Module11GlucoseExerciseAnalyzerTest -q 2>&1 | tail -10` Expected: FAIL — class does not exist

- [ ] **Step 3: Write Module11GlucoseExerciseAnalyzer implementation**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.ExerciseType;
import com.cardiofit.flink.models.GlucoseWindow;
import java.util.List;

/**
 * Exercise-specific glucose analysis for Module 11.
 *
 * Unlike Module 10's meal glucose analysis (which looks for iAUC above baseline),
 * exercise glucose analysis tracks:
 * 1. Pre-exercise glucose (baseline from last reading before activity start)
 * 2. Exercise glucose delta (mean during exercise minus pre-exercise)
 * 3. Glucose nadir (minimum during activity + 1h post-exercise recovery)
 * 4. Hypoglycemia flag (any reading < 70 mg/dL during exercise or recovery)
 * 5. Rebound hyperglycemia flag (any reading > baseline + 40 mg/dL post-exercise)
 *
 * Expected patterns:
 * - Aerobic: glucose drops 20–60 mg/dL (GLUT4 translocation)
 * - HIIT/Resistance: glucose spikes 20–40 mg/dL (catecholamine surge)
 * - Post-exercise: continued drop for 2h (glycogen resynthesis)
 *
 * Stateless utility class.
 */
public class Module11GlucoseExerciseAnalyzer {

    private static final double HYPOGLYCEMIA_THRESHOLD = 70.0;       // mg/dL
    private static final double REBOUND_HYPERGLYCEMIA_MARGIN = 40.0; // mg/dL above baseline
    private static final long ONE_HOUR_MS = 3_600_000L;

    private Module11GlucoseExerciseAnalyzer() {}
```

```java
    /**
     * Analyze glucose during an exercise session.
     *
     * @param window         glucose readings for the session
     * @param activityStart  activity start timestamp
     * @param activityEnd    activity end timestamp
     * @param exerciseType   type of exercise (affects expected pattern)
     * @return analysis result, or null if no readings
     */
    public static Result analyze(GlucoseWindow window, long activityStart, long activityEnd,
                                 ExerciseType exerciseType) {
        if (window == null || window.isEmpty()) return null;

        window.sortByTime();
        List<GlucoseWindow.Reading> readings = window.getReadings();

        // 1. Pre-exercise glucose: last reading before activity start
        Double preExerciseGlucose = null;
        for (int i = readings.size() - 1; i >= 0; i--) {
            if (readings.get(i).timestamp < activityStart) {
                preExerciseGlucose = readings.get(i).value;
                break;
            }
        }
        // Fallback to explicit baseline if no pre-exercise reading
        if (preExerciseGlucose == null) {
            preExerciseGlucose = window.getBaseline();
        }
        // Final fallback to first reading
        if (preExerciseGlucose == null) {
            preExerciseGlucose = readings.get(0).value;
        }

        // 2. Exercise glucose delta: mean during exercise minus pre-exercise
        double sumDuring = 0;
        int countDuring = 0;
        for (GlucoseWindow.Reading r : readings) {
            if (r.timestamp >= activityStart && r.timestamp <= activityEnd) {
                sumDuring += r.value;
                countDuring++;
            }
        }
        Double exerciseGlucoseDelta = null;
        if (countDuring > 0) {
            double meanDuring = sumDuring / countDuring;
            exerciseGlucoseDelta = meanDuring - preExerciseGlucose;
        }

        // 3. Glucose nadir: minimum during activity + 1h post
        double glucoseNadir = Double.MAX_VALUE;
        long nadirWindow = activityEnd + ONE_HOUR_MS;
        for (GlucoseWindow.Reading r : readings) {
            if (r.timestamp >= activityStart && r.timestamp <= nadirWindow) {
                if (r.value < glucoseNadir) {
```

```
                    glucoseNadir = r.value;
                }
            }
        }
        if (glucoseNadir == Double.MAX_VALUE) glucoseNadir = preExerciseGlucose;

        // 4. Hypoglycemia flag: any reading < 70 during exercise or recovery
        boolean hypoglycemiaFlag = false;
        for (GlucoseWindow.Reading r : readings) {
            if (r.timestamp >= activityStart && r.value < HYPOGLYCEMIA_THRESHOLD) {
                hypoglycemiaFlag = true;
                break;
            }
        }

        // 5. Rebound hyperglycemia: any reading > baseline + 40 post-exercise
        boolean reboundFlag = false;
        double reboundThreshold = preExerciseGlucose + REBOUND_HYPERGLYCEMIA_MARGIN;
        for (GlucoseWindow.Reading r : readings) {
            if (r.timestamp > activityEnd && r.value > reboundThreshold) {
                reboundFlag = true;
                break;
            }
        }

        Result result = new Result();
        result.preExerciseGlucose = preExerciseGlucose;
        result.exerciseGlucoseDelta = exerciseGlucoseDelta;
        result.glucoseNadir = glucoseNadir;
        result.hypoglycemiaFlag = hypoglycemiaFlag;
        result.reboundHyperglycemiaFlag = reboundFlag;
        result.readingCount = readings.size();
        return result;
    }

    public static class Result {
        public Double preExerciseGlucose;
        public Double exerciseGlucoseDelta;
        public double glucoseNadir;
        public boolean hypoglycemiaFlag;
        public boolean reboundHyperglycemiaFlag;
        public int readingCount;
    }
}
```

- [ ] **Step 4: Run test to verify it passes**

Run: `cd backend/shared-infrastructure/flink-processing && mvn test -pl . -Dtest=Module11GlucoseExerciseAnalyzerTest -q 2>&1 | tail -10` Expected: PASS (5 tests)

- [ ] **Step 5: Commit**

```
git add \
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/operators/Module11Gluc
  backend/shared-infrastructure/flink-processing/src/test/java/com/cardiofit/flink/operators/Module11Gluc
git commit -m "feat(module11): add Module11GlucoseExerciseAnalyzer with hypo/rebound detection — 5 tests"
```

Task 8: Create Module11ExerciseBPAnalyzer

Files: - Create: `operators/Module11ExerciseBPAnalyzer.java` - Test: `operators/Module11ExerciseBPAnalyzerTest.java`

- [ ] **Step 1: Write the failing test**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class Module11ExerciseBPAnalyzerTest {

    @Test
    void normalBPResponse_riseLessThan60() {
        Module11ExerciseBPAnalyzer.Result result =
                Module11ExerciseBPAnalyzer.analyze(120.0, 160.0, 125.0);
        assertEquals(ExerciseBPResponse.NORMAL, result.bpResponse);
        assertEquals(40.0, result.sbpRise);
        assertEquals(5.0, result.postExerciseDelta); // 125-120
    }

    @Test
    void exaggeratedResponse_riseOver60() {
        Module11ExerciseBPAnalyzer.Result result =
                Module11ExerciseBPAnalyzer.analyze(120.0, 195.0, 130.0);
        assertEquals(ExerciseBPResponse.EXAGGERATED, result.bpResponse);
        assertEquals(75.0, result.sbpRise);
        assertTrue(result.bpResponse.isPrognosticFlag());
    }

    @Test
    void exaggeratedResponse_peakOver210() {
        Module11ExerciseBPAnalyzer.Result result =
                Module11ExerciseBPAnalyzer.analyze(160.0, 215.0, 155.0);
        assertEquals(ExerciseBPResponse.EXAGGERATED, result.bpResponse);
    }

    @Test
    void postExerciseHypotension_beneficial() {
        // Post-exercise SBP drops 10 mmHg below pre (5-20 range → PEH)
        Module11ExerciseBPAnalyzer.Result result =
```

```
                Module11ExerciseBPAnalyzer.analyze(140.0, 170.0, 130.0);
        assertEquals(ExerciseBPResponse.POST_EXERCISE_HYPOTENSION, result.bpResponse);
        assertEquals(-10.0, result.postExerciseDelta);
    }

    @Test
    void hypotensiveResponse_dropOver20() {
        Module11ExerciseBPAnalyzer.Result result =
                Module11ExerciseBPAnalyzer.analyze(140.0, 170.0, 115.0);
        assertEquals(ExerciseBPResponse.HYPOTENSIVE_RESPONSE, result.bpResponse);
        assertTrue(result.bpResponse.isPrognosticFlag());
    }

    @Test
    void incomplete_missingPreExercise() {
        Module11ExerciseBPAnalyzer.Result result =
                Module11ExerciseBPAnalyzer.analyze(null, 160.0, 130.0);
        assertEquals(ExerciseBPResponse.INCOMPLETE, result.bpResponse);
    }

    @Test
    void rpp_computedCorrectly() {
        Module11ExerciseBPAnalyzer.Result result =
                Module11ExerciseBPAnalyzer.analyze(120.0, 180.0, 125.0);
        // RPP is computed externally from HR; BP analyzer provides peak SBP
        assertEquals(180.0, result.peakExerciseSBP);
    }
}
```

- [ ] **Step 2: Run test to verify it fails**

Run: `cd backend/shared-infrastructure/flink-processing && mvn test -pl . -Dtest=Module11ExerciseBPAnalyzerTest -q 2>&1 | tail -10` Expected: FAIL — class does not exist

- [ ] **Step 3: Write Module11ExerciseBPAnalyzer**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.ExerciseBPResponse;

/**
 * Exercise BP response analysis for Module 11.
 *
 * Classifies the blood pressure response during and after exercise using
 * pre-exercise, peak exercise, and post-exercise SBP readings.
 *
 * Classification priority:
 * 1. INCOMPLETE if missing pre-exercise SBP or both peak and post
 * 2. EXAGGERATED if SBP rise > 60 mmHg or peak ≥ 210 mmHg
 * 3. HYPOTENSIVE_RESPONSE if post-exercise SBP drops > 20 mmHg below pre
 * 4. POST_EXERCISE_HYPOTENSION if post-exercise SBP drops 5-20 mmHg (beneficial)
 * 5. NORMAL otherwise
```

```
 *
 * Stateless utility class.
 */
public class Module11ExerciseBPAnalyzer {

    private Module11ExerciseBPAnalyzer() {}

    /**
     * Analyze exercise BP response.
     *
     * @param preSBP   pre-exercise SBP (within 30 min before), or null
     * @param peakSBP  highest SBP during exercise, or null
     * @param postSBP  recovery SBP (5-15 min post-exercise), or null
     * @return analysis result
     */
    public static Result analyze(Double preSBP, Double peakSBP, Double postSBP) {
        Result result = new Result();
        result.preExerciseSBP = preSBP;
        result.peakExerciseSBP = peakSBP;
        result.postExerciseSBP = postSBP;

        // Classify using ExerciseBPResponse enum
        result.bpResponse = ExerciseBPResponse.classify(preSBP, peakSBP, postSBP);

        // Compute derived metrics
        if (preSBP != null && peakSBP != null) {
            result.sbpRise = peakSBP - preSBP;
        }
        if (preSBP != null && postSBP != null) {
            result.postExerciseDelta = postSBP - preSBP;
        }

        return result;
    }

    public static class Result {
        public ExerciseBPResponse bpResponse;
        public Double preExerciseSBP;
        public Double peakExerciseSBP;
        public Double postExerciseSBP;
        public Double sbpRise;
        public Double postExerciseDelta;
    }
}
```

- [ ] **Step 4: Run test to verify it passes**

Run: `cd backend/shared-infrastructure/flink-processing && mvn test -pl . -Dtest=Module11ExerciseBPAnalyzerTest -q 2>&1 | tail -10` Expected: PASS (7 tests)

- [ ] **Step 5: Commit**

```
git add \
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/operators/Module11Exer
  backend/shared-infrastructure/flink-processing/src/test/java/com/cardiofit/flink/operators/Module11Exer
git commit -m "feat(module11): add Module11ExerciseBPAnalyzer with exaggerated/PEH classification — 7 tes
```

Task 9: Create Module11_ActivityResponseCorrelator (Main KPF)

Files: - Create: `operators/Module11_ActivityResponseCorrelator.java`

- [ ] **Step 1: Create Module11_ActivityResponseCorrelator**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.*;
import org.apache.flink.api.common.functions.OpenContext;
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
import org.apache.flink.api.common.state.StateTtlConfig;
import org.apache.flink.api.common.typeinfo.TypeInformation;
import org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import org.apache.flink.util.Collector;
import org.apache.flink.util.OutputTag;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.time.Duration;
import java.util.List;
import java.util.Map;
import java.util.UUID;

/**
 * Module 11: Activity Response Correlator — main operator.
 *
 * Exercise-session-window-driven KeyedProcessFunction:
 * - ACTIVITY event (PATIENT_REPORTED + report_type=ACTIVITY_LOG) → OPENS session
 * - DEVICE_READING (HR) → FILLS the HR window
 * - DEVICE_READING (CGM) or LAB_RESULT (glucose) → FILLS glucose window
 * - VITAL_SIGN (BP) → Captures pre/peak/post BP + buffers for future sessions
 * - VITAL_SIGN (resting_hr) → Updates resting HR baseline
 * - Processing-time timer at activity_end + 2h + 5min → CLOSES and emits
 *
 * Session window duration: activity_duration + 2h recovery + 5min grace.
 * Capped at 6h05m total. Default activity duration: 30 min if unspecified.
 *
 * Keyed by patientId. Input: CanonicalEvent from enriched-patient-events-v1.
 * Output: ActivityResponseRecord to flink.activity-response (main output).
 *
 * State TTL: 7 days.
 */
public class Module11_ActivityResponseCorrelator
```

```
    extends KeyedProcessFunction<String, CanonicalEvent, ActivityResponseRecord> {

private static final Logger LOG =
        LoggerFactory.getLogger(Module11_ActivityResponseCorrelator.class);

public static final OutputTag<ActivityResponseRecord> FITNESS_PATTERN_FEED_TAG =
        new OutputTag<>("fitness-pattern-feed",
                TypeInformation.of(ActivityResponseRecord.class));

private transient ValueState<ActivityCorrelationState> correlationState;

@Override
public void open(OpenContext openContext) throws Exception {
    super.open(openContext);

    ValueStateDescriptor<ActivityCorrelationState> stateDesc =
            new ValueStateDescriptor<>("activity-correlation-state", ActivityCorrelationState.class);

    StateTtlConfig ttl = StateTtlConfig
            .newBuilder(Duration.ofDays(7))
            .setUpdateType(StateTtlConfig.UpdateType.OnReadAndWrite)
            .setStateVisibility(StateTtlConfig.StateVisibility.NeverReturnExpired)
            .build();
    stateDesc.enableTimeToLive(ttl);
    correlationState = getRuntimeContext().getState(stateDesc);

    LOG.info("Module 11 Activity Response Correlator initialized (exercise-session-window, 3-phase HR
}

@Override
public void processElement(CanonicalEvent event, Context ctx,
                           Collector<ActivityResponseRecord> out) throws Exception {
    ActivityCorrelationState state = correlationState.value();
    if (state == null) {
        state = new ActivityCorrelationState(event.getPatientId());
    }

    EventType eventType = event.getEventType();
    Map<String, Object> payload = event.getPayload();
    if (payload == null) {
        correlationState.update(state);
        return;
    }

    // Extract patient age if available (for HR_max calculation)
    if (state.getPatientAge() == null) {
        Object ageObj = payload.get("patient_age");
        if (ageObj instanceof Number) {
            state.setPatientAge(((Number) ageObj).intValue());
        }
    }

    // Route by event type
    if (isActivityEvent(eventType, payload)) {
```

```
            handleActivityEvent(event, state, ctx);
        } else if (isHRReading(eventType, payload)) {
            handleHRReading(event, state);
        } else if (isGlucoseReading(eventType, payload)) {
            handleGlucoseReading(event, state);
        } else if (eventType == EventType.VITAL_SIGN) {
            handleVitalSign(event, state);
        }

        state.setLastUpdated(ctx.timerService().currentProcessingTime());
        correlationState.update(state);
    }

    @Override
    public void onTimer(long timestamp, OnTimerContext ctx,
                        Collector<ActivityResponseRecord> out) throws Exception {
        ActivityCorrelationState state = correlationState.value();
        if (state == null) return;

        List<String> sessionIds = state.getSessionsForTimer(timestamp);
        for (String activityEventId : sessionIds) {
            ActivityCorrelationState.ActivitySession session = state.closeSession(activityEventId);
            if (session == null) continue;

            ActivityResponseRecord record = buildRecord(state, session);
            out.collect(record);

            ctx.output(FITNESS_PATTERN_FEED_TAG, record);

            LOG.debug("Activity session closed: patient={}, activity={}, type={}, peakHR={}, hrrClass={}"
                    state.getPatientId(), activityEventId, session.exerciseType,
                    record.getPeakHR(), record.getHrRecoveryClass());
        }

        correlationState.update(state);
    }

    // --- Event Classification ---

    private boolean isActivityEvent(EventType type, Map<String, Object> payload) {
        if (type != EventType.PATIENT_REPORTED) return false;
        Object reportType = payload.get("report_type");
        return "ACTIVITY_LOG".equalsIgnoreCase(reportType != null ? reportType.toString() : "");
    }

    private boolean isHRReading(EventType type, Map<String, Object> payload) {
        if (type != EventType.DEVICE_READING) return false;
        return payload.containsKey("heart_rate");
    }

    private boolean isGlucoseReading(EventType type, Map<String, Object> payload) {
        if (type == EventType.DEVICE_READING) {
            return payload.containsKey("glucose_value");
        }
```

```
        if (type == EventType.LAB_RESULT) {
            Object labType = payload.get("lab_type");
            return "glucose".equalsIgnoreCase(labType != null ? labType.toString() : "");
        }
        return false;
    }

    // --- Event Handlers ---

    private void handleActivityEvent(CanonicalEvent event, ActivityCorrelationState state, Context ctx) {
        String activityEventId = event.getId() != null ? event.getId() : UUID.randomUUID().toString();
        long activityStart = event.getEventTime();
        Map<String, Object> payload = event.getPayload();

        // Parse duration from payload
        long durationMs = 30L * 60_000L; // default 30 min
        Object durationObj = payload.get("duration_minutes");
        if (durationObj instanceof Number) {
            durationMs = ((Number) durationObj).longValue() * 60_000L;
        }

        long timerFireTime = state.openSession(activityEventId, activityStart, durationMs, payload);
        ctx.timerService().registerProcessingTimeTimer(timerFireTime);

        LOG.debug("Activity session opened: patient={}, activity={}, type={}, duration={}min, timerAt={}"
                state.getPatientId(), activityEventId,
                ExerciseType.fromString(payload.get("exercise_type") != null ? payload.get("exercise_type
                durationMs / 60_000L, timerFireTime);
    }

    private void handleHRReading(CanonicalEvent event, ActivityCorrelationState state) {
        Map<String, Object> payload = event.getPayload();
        Object hrObj = payload.get("heart_rate");
        if (!(hrObj instanceof Number)) return;
        double heartRate = ((Number) hrObj).doubleValue();
        String source = payload.containsKey("source") ? payload.get("source").toString() : "WEARABLE";

        // Check if this is a resting HR reading
        Object activityFlag = payload.get("activity_state");
        if ("RESTING".equalsIgnoreCase(activityFlag != null ? activityFlag.toString() : "")) {
            state.updateRestingHR(heartRate, event.getEventTime());
        }

        state.addHRReading(event.getEventTime(), heartRate, source);
    }

    private void handleGlucoseReading(CanonicalEvent event, ActivityCorrelationState state) {
        Map<String, Object> payload = event.getPayload();
        double glucoseValue;
        String source;

        if (event.getEventType() == EventType.DEVICE_READING) {
            Object val = payload.get("glucose_value");
            if (!(val instanceof Number)) return;
```

```
            glucoseValue = ((Number) val).doubleValue();
            source = "CGM";
        } else {
            Object val = payload.get("value");
            if (!(val instanceof Number)) return;
            glucoseValue = ((Number) val).doubleValue();
            source = "SMBG";
        }

        state.addGlucoseReading(event.getEventTime(), glucoseValue, source);
    }

    private void handleVitalSign(CanonicalEvent event, ActivityCorrelationState state) {
        Map<String, Object> payload = event.getPayload();

        // BP reading
        Object sbpObj = payload.get("systolic_bp");
        if (sbpObj instanceof Number) {
            double sbp = ((Number) sbpObj).doubleValue();
            double dbp = (payload.get("diastolic_bp") instanceof Number)
                    ? ((Number) payload.get("diastolic_bp")).doubleValue() : 0.0;
            state.addBPReading(event.getEventTime(), sbp, dbp);
        }

        // Resting HR from vital sign
        Object restingHR = payload.get("resting_heart_rate");
        if (restingHR instanceof Number) {
            state.updateRestingHR(((Number) restingHR).doubleValue(), event.getEventTime());
        }
    }

    // --- Record Building ---

    private ActivityResponseRecord buildRecord(ActivityCorrelationState state,
                                               ActivityCorrelationState.ActivitySession session) {
        // Analyze HR
        Module11HRAnalyzer.Result hrResult = Module11HRAnalyzer.analyze(session.hrWindow);

        // Analyze glucose
        Module11GlucoseExerciseAnalyzer.Result glucoseResult =
                Module11GlucoseExerciseAnalyzer.analyze(
                        session.glucoseWindow,
                        session.activityStartTime,
                        session.activityEndTime != null ? session.activityEndTime : session.activityStart
                        session.exerciseType);

        // Analyze BP
        Double preSBP = session.bpWindow.getPreMealSBP(); // reusing BPWindow (pre-meal = pre-exercise)
        Double peakSBP = session.peakExerciseSBP;
        Double postSBP = session.bpWindow.getPostMealSBP();
        Module11ExerciseBPAnalyzer.Result bpResult =
                Module11ExerciseBPAnalyzer.analyze(preSBP, peakSBP, postSBP);

        // Compute RPP
```

```
        Double peakRPP = null;
        if (hrResult != null && peakSBP != null) {
            peakRPP = Module11HRAnalyzer.computeRPP(hrResult.peakHR, peakSBP);
        }

        // MET-minutes
        double durationMin = session.reportedDurationMs / 60_000.0;
        double metMinutes = session.reportedMETs * durationMin;

        // Quality score
        double qualityScore = computeQualityScore(hrResult, glucoseResult, bpResult);

        // Window duration: use deterministic value from session boundaries
        long windowDurationMs = session.timerFireTime - session.activityStartTime;

        ActivityResponseRecord.Builder builder = ActivityResponseRecord.builder()
                .recordId("m11-" + UUID.randomUUID())
                .patientId(state.getPatientId())
                .activityEventId(session.activityEventId)
                .activityStartTime(session.activityStartTime)
                .activityDurationMin(durationMin)
                .exerciseType(session.exerciseType)
                .reportedMETs(session.reportedMETs)
                .metMinutes(metMinutes)
                .concurrent(session.concurrent)
                .windowDurationMs(windowDurationMs)
                .qualityScore(qualityScore);

        // HR features
        if (hrResult != null) {
            builder.peakHR(hrResult.peakHR)
                    .meanActiveHR(hrResult.meanActiveHR)
                    .hrr1(hrResult.hrr1)
                    .hrr2(hrResult.hrr2)
                    .hrRecoveryClass(hrResult.hrRecoveryClass)
                    .dominantZone(hrResult.dominantZone)
                    .hrReadingCount(hrResult.readingCount);
        }
        builder.restingHR(state.getLastRestingHR());

        // Glucose features
        if (glucoseResult != null) {
            builder.preExerciseGlucose(glucoseResult.preExerciseGlucose)
                    .exerciseGlucoseDelta(glucoseResult.exerciseGlucoseDelta)
                    .glucoseNadir(glucoseResult.glucoseNadir)
                    .hypoglycemiaFlag(glucoseResult.hypoglycemiaFlag)
                    .reboundHyperglycemiaFlag(glucoseResult.reboundHyperglycemiaFlag);
        }

        // BP features
        builder.preExerciseSBP(bpResult.preExerciseSBP)
                .peakExerciseSBP(bpResult.peakExerciseSBP)
                .postExerciseSBP(bpResult.postExerciseSBP)
                .exerciseBPResponse(bpResult.bpResponse)
```

```
                .peakRPP(peakRPP);

        return builder.build();
    }

    private double computeQualityScore(Module11HRAnalyzer.Result hr,
                                       Module11GlucoseExerciseAnalyzer.Result glucose,
                                       Module11ExerciseBPAnalyzer.Result bp) {
        double score = 0.0;
        if (hr != null) {
            score += 0.4 * hr.qualityScore;
            if (hr.hrr1 != null) score += 0.15; // HRR available adds quality
        }
        if (glucose != null && glucose.readingCount > 0) {
            score += 0.25;
        }
        if (bp != null && bp.bpResponse != ExerciseBPResponse.INCOMPLETE) {
            score += 0.2;
        }
        return Math.min(1.0, score);
    }
}
```

- [ ] **Step 2: Compile**

Run: `cd backend/shared-infrastructure/flink-processing && mvn compile -pl . -q 2>&1 | tail -5`
Expected: BUILD SUCCESS

- [ ] **Step 3: Commit**

```
git add backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/operators/Module
git commit -m "feat(module11): add Module11_ActivityResponseCorrelator main KPF with exercise session win
```

Task 10: Create Module 11b Operators

Files: - Create: `operators/Module11bVO2maxEstimator.java` - Create: `operators/Module11bExerciseDoseCalculator.java` - Test: `operators/Module11bVO2maxEstimatorTest.java` - Test: `operators/Module11bExerciseDoseCalculatorTest.java`

- [ ] **Step 1: Write VO2max estimator failing test**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.*;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;
```

```
class Module11bVO2maxEstimatorTest {

    @Test
    void estimateFromSubmaximalHR_healthyAdult() {
        // Age 40, resting HR 65, peak exercise HR 155, HR_max = 208-0.7*40 = 180
        // VO2max = 15 × (HR_max / HR_rest) = 15 × (180/65) ≈ 41.5
        // More accurate: extrapolate from submaximal
        Module11bVO2maxEstimator.Result result =
                Module11bVO2maxEstimator.estimate(155.0, 65.0, 180.0);
        assertNotNull(result);
        assertTrue(result.vo2max > 30.0 && result.vo2max < 55.0);
        assertNotNull(result.fitnessLevel);
    }

    @Test
    void estimateFromSubmaximalHR_highFitness() {
        // Athlete: resting HR 50, peak HR 175, HR_max 190
        Module11bVO2maxEstimator.Result result =
                Module11bVO2maxEstimator.estimate(175.0, 50.0, 190.0);
        assertTrue(result.vo2max > 40.0);
        assertTrue(result.fitnessLevel == FitnessLevel.GOOD
                || result.fitnessLevel == FitnessLevel.EXCELLENT);
    }

    @Test
    void estimateFromSubmaximalHR_deconditoned() {
        // Sedentary: resting HR 85, peak HR 140, HR_max 175
        Module11bVO2maxEstimator.Result result =
                Module11bVO2maxEstimator.estimate(140.0, 85.0, 175.0);
        assertTrue(result.vo2max < 40.0);
    }

    @Test
    void nullRestingHR_usesDefault() {
        Module11bVO2maxEstimator.Result result =
                Module11bVO2maxEstimator.estimate(160.0, null, 180.0);
        assertNotNull(result);
        // Uses default resting HR of 72
    }

    @Test
    void insufficientData_peakHRTooLow() {
        // Peak HR below 60% of HR_max — not valid submaximal test
        Module11bVO2maxEstimator.Result result =
                Module11bVO2maxEstimator.estimate(95.0, 70.0, 180.0);
        assertNull(result); // 95/180 = 53% < 60% threshold
    }

    @Test
    void averageFromMultipleSessions() {
        List<Double> peakHRs = List.of(155.0, 160.0, 158.0);
        Double restingHR = 65.0;
        double hrMax = 180.0;
```

```
        List<Module11bVO2maxEstimator.Result> results = new ArrayList<>();
        for (Double peakHR : peakHRs) {
            Module11bVO2maxEstimator.Result r =
                    Module11bVO2maxEstimator.estimate(peakHR, restingHR, hrMax);
            if (r != null) results.add(r);
        }
        assertEquals(3, results.size());
        // Averaging produces more stable estimate
        double avgVO2max = results.stream().mapToDouble(r -> r.vo2max).average().orElse(0);
        assertTrue(avgVO2max > 30.0);
    }
}
```

- [ ] **Step 2: Run test to verify it fails**

Run: `cd backend/shared-infrastructure/flink-processing && mvn test -pl . -Dtest=Module11bVO2maxEstimatorTest -q 2>&1 | tail -10` Expected: FAIL — class does not exist

- [ ] **Step 3: Write Module11bVO2maxEstimator**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.FitnessLevel;

/**
 * Submaximal VO2max estimation for Module 11b.
 *
 * Uses the ACSM submaximal extrapolation method:
 *   VO2max = VO2_submaximal × (HR_max - HR_rest) / (HR_exercise - HR_rest)
 *
 * Where VO2_submaximal is estimated from the known linear relationship
 * between HR and VO2 during steady-state aerobic exercise:
 *   VO2_submaximal = (HR_exercise - HR_rest) / (HR_max - HR_rest) × VO2max
 *
 * Simplified Astrand-Ryhming approach for streaming context:
 *   VO2max ≈ 15 × (HR_max / HR_rest)  [Uth et al., Eur J Appl Physiol 2004]
 *
 * This module uses a hybrid: the Astrand-Ryhming baseline adjusted by the
 * submaximal exercise HR fraction to improve accuracy.
 *
 * Validation constraints:
 * - Peak HR must be ≥ 60% of HR_max (submaximal effort threshold)
 * - Peak HR must be ≤ HR_max (physiological ceiling)
 * - Resting HR defaults to 72 bpm if unavailable
 *
 * Accuracy: ±10-15% vs direct VO2max measurement. Improves with
 * multiple sessions (averaging reduces noise from day-to-day HR variability).
 */
public class Module11bVO2maxEstimator {

    private static final double DEFAULT_RESTING_HR = 72.0;
```

```
    private static final double MIN_EFFORT_FRACTION = 0.60; // 60% of HR_max minimum

    private Module11bVO2maxEstimator() {}

    /**
     * Estimate VO2max from submaximal exercise HR data.
     *
     * @param peakExerciseHR peak HR observed during exercise
     * @param restingHR      resting HR (or null for default)
     * @param hrMax          age-predicted HR_max
     * @return result with VO2max estimate and fitness level, or null if insufficient effort
     */
    public static Result estimate(Double peakExerciseHR, Double restingHR, double hrMax) {
        if (peakExerciseHR == null || hrMax <= 0) return null;

        double rhr = (restingHR != null && restingHR > 30) ? restingHR : DEFAULT_RESTING_HR;

        // Validate minimum effort
        if (peakExerciseHR / hrMax < MIN_EFFORT_FRACTION) return null;
        if (peakExerciseHR > hrMax) peakExerciseHR = hrMax; // cap at HR_max

        // Astrand-Ryhming estimation: VO2max = 15 × (HR_max / HR_rest)
        double astrand = 15.0 * (hrMax / rhr);

        // Submaximal adjustment: scale by effort fraction using HR reserve
        double hrReserve = hrMax - rhr;
        if (hrReserve <= 0) hrReserve = 1.0; // guard
        double effortFraction = (peakExerciseHR - rhr) / hrReserve;

        // Hybrid estimate: Astrand base adjusted by observed effort
        // Higher effort fraction → more reliable estimate → less correction needed
        double vo2max = astrand * (0.8 + 0.2 * effortFraction);

        // Clamp to physiological range
        vo2max = Math.max(8.0, Math.min(80.0, vo2max));

        Result result = new Result();
        result.vo2max = vo2max;
        result.fitnessLevel = FitnessLevel.fromVO2max(vo2max);
        result.effortFraction = effortFraction;
        return result;
    }

    public static class Result {
        public double vo2max;
        public FitnessLevel fitnessLevel;
        public double effortFraction;
    }
}
```

- [ ] **Step 4: Run VO2max tests**

Run: `cd backend/shared-infrastructure/flink-processing && mvn test -pl . -Dtest=Module11bVO2maxEstimatorTest -q 2>&1 | tail -10` Expected: PASS (6 tests)

- [ ] **Step 5: Write exercise dose calculator failing test**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.*;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

class Module11bExerciseDoseCalculatorTest {

    @Test
    void weeklyDose_sumsMetMinutes() {
        List<ActivityResponseRecord> records = new ArrayList<>();
        records.add(activityRecord(ExerciseType.AEROBIC, 6.0, 30.0)); // 180 MET-min
        records.add(activityRecord(ExerciseType.AEROBIC, 6.0, 45.0)); // 270 MET-min
        records.add(activityRecord(ExerciseType.RESISTANCE, 5.0, 40.0)); // 200 MET-min

        Module11bExerciseDoseCalculator.Result result =
                Module11bExerciseDoseCalculator.calculate(records);

        assertEquals(650.0, result.totalMetMinutes, 0.1);
        assertEquals(115.0, result.totalDurationMin, 0.1);
        assertTrue(result.meetsWHOModerate); // 650 >= 150 MET-min
    }

    @Test
    void whoBenchmark_belowMinimum() {
        List<ActivityResponseRecord> records = new ArrayList<>();
        records.add(activityRecord(ExerciseType.FLEXIBILITY, 2.5, 30.0)); // 75 MET-min

        Module11bExerciseDoseCalculator.Result result =
                Module11bExerciseDoseCalculator.calculate(records);

        assertEquals(75.0, result.totalMetMinutes, 0.1);
        assertFalse(result.meetsWHOModerate); // 75 < 150
    }

    @Test
    void emptyRecords_zeroResult() {
        Module11bExerciseDoseCalculator.Result result =
                Module11bExerciseDoseCalculator.calculate(new ArrayList<>());

        assertEquals(0.0, result.totalMetMinutes);
        assertEquals(0, result.activityCount);
        assertFalse(result.meetsWHOModerate);
    }

    @Test
```

```
    void perTypeBreakdown_correct() {
        List<ActivityResponseRecord> records = new ArrayList<>();
        records.add(activityRecord(ExerciseType.AEROBIC, 6.0, 30.0));
        records.add(activityRecord(ExerciseType.AEROBIC, 7.0, 45.0));
        records.add(activityRecord(ExerciseType.RESISTANCE, 5.0, 40.0));

        Module11bExerciseDoseCalculator.Result result =
                Module11bExerciseDoseCalculator.calculate(records);

        assertEquals(2, result.perTypeMetMinutes.get(ExerciseType.AEROBIC).intValue() > 0 ? 2 : 0);
        assertTrue(result.perTypeMetMinutes.containsKey(ExerciseType.AEROBIC));
        assertTrue(result.perTypeMetMinutes.containsKey(ExerciseType.RESISTANCE));
    }

    private ActivityResponseRecord activityRecord(ExerciseType type, double mets, double durationMin) {
        return ActivityResponseRecord.builder()
                .recordId("test-" + System.nanoTime())
                .patientId("P1")
                .activityEventId("act-" + System.nanoTime())
                .activityStartTime(System.currentTimeMillis())
                .exerciseType(type)
                .reportedMETs(mets)
                .activityDurationMin(durationMin)
                .metMinutes(mets * durationMin)
                .build();
    }
}
```

- [ ] **Step 6: Write Module11bExerciseDoseCalculator**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.ActivityResponseRecord;
import com.cardiofit.flink.models.ExerciseType;

import java.util.*;

/**
 * Exercise dose calculator for Module 11b.
 *
 * Computes weekly exercise dose metrics:
 * - Total MET-minutes (sum of METs × duration for all activities)
 * - Total active duration
 * - Per-exercise-type breakdown
 * - WHO guideline adherence check
 *
 * WHO Physical Activity Guidelines (2020):
 * - Moderate: 150–300 MET-minutes/week (e.g., 150 min walking at 3.3 METs ≈ 495)
 * - Vigorous: 75–150 MET-minutes/week (e.g., 75 min running at 8 METs = 600)
 * - Combined: ≥ 600 MET-minutes/week for substantial benefit
 *
 * Simplified benchmark: ≥ 150 MET-minutes/week = meets minimum.
 */
```

```
public class Module11bExerciseDoseCalculator {

    private static final double WHO_MODERATE_MINIMUM = 150.0; // MET-minutes/week

    private Module11bExerciseDoseCalculator() {}

    public static Result calculate(List<ActivityResponseRecord> records) {
        Result result = new Result();
        if (records == null || records.isEmpty()) {
            result.perTypeMetMinutes = Collections.emptyMap();
            return result;
        }

        double totalMetMin = 0;
        double totalDuration = 0;
        Map<ExerciseType, Double> perType = new LinkedHashMap<>();

        for (ActivityResponseRecord r : records) {
            double mm = r.getMetMinutes();
            totalMetMin += mm;
            totalDuration += r.getActivityDurationMin();
            perType.merge(r.getExerciseType(), mm, Double::sum);
        }

        result.totalMetMinutes = totalMetMin;
        result.totalDurationMin = totalDuration;
        result.activityCount = records.size();
        result.perTypeMetMinutes = perType;
        result.meetsWHOModerate = totalMetMin >= WHO_MODERATE_MINIMUM;

        return result;
    }

    public static class Result {
        public double totalMetMinutes;
        public double totalDurationMin;
        public int activityCount;
        public Map<ExerciseType, Double> perTypeMetMinutes;
        public boolean meetsWHOModerate;
    }
}
```

- [ ] **Step 7: Run exercise dose tests**

Run: `cd backend/shared-infrastructure/flink-processing && mvn test -pl . -Dtest=Module11bExerciseDoseCalculatorTest -q 2>&1 | tail -10` Expected: PASS (4 tests)

- [ ] **Step 8: Commit**

```
git add \
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/operators/Module11bVO2
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/operators/Module11bExe
```

```
  backend/shared-infrastructure/flink-processing/src/test/java/com/cardiofit/flink/operators/Module11bVO2
  backend/shared-infrastructure/flink-processing/src/test/java/com/cardiofit/flink/operators/Module11bExe
git commit -m "feat(module11b): add VO2maxEstimator (Astrand-Ryhming hybrid) and ExerciseDoseCalculator w
```

Task 11: Create Module11b_FitnessPatternAggregator

Files: - Create: `operators/Module11b_FitnessPatternAggregator.java`

- [ ] **Step 1: Create Module11b_FitnessPatternAggregator**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.*;
import org.apache.flink.api.common.functions.OpenContext;
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
import org.apache.flink.api.common.state.StateTtlConfig;
import org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import org.apache.flink.util.Collector;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.time.*;
import java.time.temporal.TemporalAdjusters;
import java.util.*;
import java.util.stream.Collectors;

/**
 * Module 11b: Fitness Pattern Aggregator — weekly KPF.
 *
 * Consumes ActivityResponseRecord (output of Module 11).
 * Accumulates records for 7 days, then on weekly timer (Monday 00:00 UTC):
 * 1. Computes exercise dose (total MET-minutes, WHO adherence)
 * 2. Aggregates HR metrics (mean peak HR, mean HRR₁, dominant recovery class)
 * 3. Estimates VO2max from submaximal exercise data
 * 4. Computes VO2max trend (slope over 90-day rolling buffer)
 * 5. Computes zone distribution across all sessions
 * 6. Aggregates glucose-exercise response
 * 7. Emits FitnessPatternSummary
 *
 * Separate Flink job from Module 11 for failure isolation.
 * Input: ActivityResponseRecord from flink.activity-response
 * Output: FitnessPatternSummary to flink.fitness-patterns
 *
 * State TTL: 90 days (VO2max trend buffer).
 */
public class Module11b_FitnessPatternAggregator
        extends KeyedProcessFunction<String, ActivityResponseRecord, FitnessPatternSummary> {

    private static final Logger LOG =
```

```
        LoggerFactory.getLogger(Module11b_FitnessPatternAggregator.class);

    private transient ValueState<FitnessPatternState> patternState;

    @Override
    public void open(OpenContext openContext) throws Exception {
        super.open(openContext);

        ValueStateDescriptor<FitnessPatternState> stateDesc =
                new ValueStateDescriptor<>("fitness-pattern-state", FitnessPatternState.class);

        StateTtlConfig ttl = StateTtlConfig
                .newBuilder(Duration.ofDays(90))
                .setUpdateType(StateTtlConfig.UpdateType.OnReadAndWrite)
                .setStateVisibility(StateTtlConfig.StateVisibility.NeverReturnExpired)
                .build();
        stateDesc.enableTimeToLive(ttl);
        patternState = getRuntimeContext().getState(stateDesc);

        LOG.info("Module 11b Fitness Pattern Aggregator initialized (weekly, 90d VO2max trend buffer)");
    }

    @Override
    public void processElement(ActivityResponseRecord record, Context ctx,
                               Collector<FitnessPatternSummary> out) throws Exception {
        FitnessPatternState state = patternState.value();
        if (state == null) {
            state = new FitnessPatternState(record.getPatientId());
        }

        state.addActivityRecord(record);

        // Update resting HR if available
        if (record.getRestingHR() != null) {
            state.setLastKnownRestingHR(record.getRestingHR());
        }

        // Register weekly timer (Monday 00:00 UTC) — once per patient
        if (!state.isWeeklyTimerRegistered()) {
            long nextMonday = computeNextMonday(ctx.timerService().currentProcessingTime());
            ctx.timerService().registerProcessingTimeTimer(nextMonday);
            state.setWeeklyTimerRegistered(true);
        }

        state.setLastUpdated(ctx.timerService().currentProcessingTime());
        patternState.update(state);
    }

    @Override
    public void onTimer(long timestamp, OnTimerContext ctx,
                        Collector<FitnessPatternSummary> out) throws Exception {
        FitnessPatternState state = patternState.value();
        if (state == null) return;
```

```
        List<ActivityResponseRecord> weeklyRecords = state.drainWeeklyRecords();
        if (!weeklyRecords.isEmpty()) {
            FitnessPatternSummary summary = buildSummary(state, weeklyRecords, timestamp);
            out.collect(summary);
            LOG.info("Weekly fitness pattern emitted: patient={}, activities={}, metMin={:.0f}, vo2max={}
                    state.getPatientId(), weeklyRecords.size(),
                    summary.getTotalMetMinutes(), summary.getEstimatedVO2max(),
                    summary.getFitnessLevel());
        }

        state.setLastWeeklyEmitTimestamp(timestamp);

        // Re-register next Monday
        long nextMonday = computeNextMonday(timestamp);
        ctx.timerService().registerProcessingTimeTimer(nextMonday);
        patternState.update(state);
    }

    private FitnessPatternSummary buildSummary(FitnessPatternState state,
                                               List<ActivityResponseRecord> records,
                                               long timestamp) {
        FitnessPatternSummary summary = new FitnessPatternSummary();
        summary.setSummaryId("m11b-" + UUID.randomUUID());
        summary.setPatientId(state.getPatientId());

        // Period range
        long minTs = records.stream().mapToLong(ActivityResponseRecord::getActivityStartTime).min().orEls
        long maxTs = records.stream().mapToLong(ActivityResponseRecord::getActivityStartTime).max().orEls
        summary.setPeriodStartMs(minTs);
        summary.setPeriodEndMs(maxTs);

        // Exercise dose
        Module11bExerciseDoseCalculator.Result doseResult =
                Module11bExerciseDoseCalculator.calculate(records);
        summary.setTotalMetMinutes(doseResult.totalMetMinutes);
        summary.setTotalActiveDurationMin(doseResult.totalDurationMin);
        summary.setActivityCount(doseResult.activityCount);

        // HR aggregation
        List<ActivityResponseRecord> withHR = records.stream()
                .filter(r -> r.getPeakHR() != null)
                .collect(Collectors.toList());
        summary.setSessionsWithHR(withHR.size());

        if (!withHR.isEmpty()) {
            summary.setMeanPeakHR(withHR.stream()
                    .mapToDouble(ActivityResponseRecord::getPeakHR)
                    .average().orElse(0));

            List<ActivityResponseRecord> withHRR = withHR.stream()
                    .filter(r -> r.getHrr1() != null)
                    .collect(Collectors.toList());
            if (!withHRR.isEmpty()) {
                summary.setMeanHRR1(withHRR.stream()
```

```
                    .mapToDouble(ActivityResponseRecord::getHrr1)
                    .average().orElse(0));
        }

        // Dominant HRR class (mode)
        Map<HRRecoveryClass, Long> hrrCounts = withHR.stream()
                .filter(r -> r.getHrRecoveryClass() != null
                        && r.getHrRecoveryClass() != HRRecoveryClass.INSUFFICIENT_DATA)
                .collect(Collectors.groupingBy(ActivityResponseRecord::getHrRecoveryClass, Collectors
        if (!hrrCounts.isEmpty()) {
            summary.setDominantHRRecoveryClass(
                    hrrCounts.entrySet().stream()
                            .max(Map.Entry.comparingByValue())
                            .get().getKey());
        }

        // Zone distribution (percentage across all sessions)
        Map<ActivityIntensityZone, Double> zonePct = new EnumMap<>(ActivityIntensityZone.class);
        // This would require zone data from each session; use dominant zone as proxy
        Map<ActivityIntensityZone, Long> zoneCounts = withHR.stream()
                .filter(r -> r.getDominantZone() != null)
                .collect(Collectors.groupingBy(ActivityResponseRecord::getDominantZone, Collectors.co
        long totalZoneSessions = zoneCounts.values().stream().mapToLong(Long::longValue).sum();
        for (Map.Entry<ActivityIntensityZone, Long> entry : zoneCounts.entrySet()) {
            zonePct.put(entry.getKey(), (double) entry.getValue() / totalZoneSessions * 100.0);
        }
        summary.setZoneDistributionPct(zonePct);
    }

    // VO2max estimation (average from sessions with sufficient effort)
    if (withHR.size() >= FitnessLevel.MIN_SESSIONS_FOR_ESTIMATION) {
        List<Double> vo2maxEstimates = new ArrayList<>();
        for (ActivityResponseRecord r : withHR) {
            Module11bVO2maxEstimator.Result vo2Result =
                    Module11bVO2maxEstimator.estimate(
                            r.getPeakHR(), state.getLastKnownRestingHR(), state.getHrMax());
            if (vo2Result != null) {
                vo2maxEstimates.add(vo2Result.vo2max);
            }
        }
        if (!vo2maxEstimates.isEmpty()) {
            double avgVO2max = vo2maxEstimates.stream().mapToDouble(Double::doubleValue).average().or
            summary.setEstimatedVO2max(avgVO2max);
            summary.setFitnessLevel(FitnessLevel.fromVO2max(avgVO2max));

            // Store in rolling buffer for trend
            state.addVO2maxEstimate(avgVO2max, timestamp);
            summary.setVo2maxTrend(state.computeVO2maxTrendPerWeek());
        }
    } else {
        summary.setFitnessLevel(FitnessLevel.INSUFFICIENT_DATA);
    }

    // Glucose-exercise response aggregation
```

```
        List<ActivityResponseRecord> withGlucose = records.stream()
                .filter(r -> r.getExerciseGlucoseDelta() != null)
                .collect(Collectors.toList());
        summary.setSessionsWithGlucose(withGlucose.size());

        if (!withGlucose.isEmpty()) {
            summary.setMeanExerciseGlucoseDelta(withGlucose.stream()
                    .mapToDouble(ActivityResponseRecord::getExerciseGlucoseDelta)
                    .average().orElse(0));

            summary.setHypoglycemiaEventCount((int) records.stream()
                    .filter(ActivityResponseRecord::isHypoglycemiaFlag).count());

            // Mean glucose drop for aerobic sessions specifically
            List<ActivityResponseRecord> aerobicWithGlucose = withGlucose.stream()
                    .filter(r -> r.getExerciseType() == ExerciseType.AEROBIC)
                    .collect(Collectors.toList());
            if (!aerobicWithGlucose.isEmpty()) {
                summary.setMeanGlucoseDropAerobic(aerobicWithGlucose.stream()
                        .mapToDouble(ActivityResponseRecord::getExerciseGlucoseDelta)
                        .average().orElse(0));
            }
        }

        // Exercise type breakdown
        Map<ExerciseType, FitnessPatternSummary.ExerciseTypeStats> breakdown = new LinkedHashMap<>();
        Map<ExerciseType, List<ActivityResponseRecord>> byType = records.stream()
                .collect(Collectors.groupingBy(ActivityResponseRecord::getExerciseType));
        for (Map.Entry<ExerciseType, List<ActivityResponseRecord>> entry : byType.entrySet()) {
            FitnessPatternSummary.ExerciseTypeStats stats = new FitnessPatternSummary.ExerciseTypeStats()
            List<ActivityResponseRecord> activities = entry.getValue();
            stats.sessionCount = activities.size();
            stats.totalMetMinutes = activities.stream().mapToDouble(ActivityResponseRecord::getMetMinutes
            stats.meanPeakHR = activities.stream()
                    .filter(r -> r.getPeakHR() != null)
                    .mapToDouble(ActivityResponseRecord::getPeakHR)
                    .average().orElse(0);
            List<ActivityResponseRecord> withGluc = activities.stream()
                    .filter(r -> r.getExerciseGlucoseDelta() != null)
                    .collect(Collectors.toList());
            if (!withGluc.isEmpty()) {
                stats.meanGlucoseDelta = withGluc.stream()
                        .mapToDouble(ActivityResponseRecord::getExerciseGlucoseDelta)
                        .average().orElse(0);
            }
            breakdown.put(entry.getKey(), stats);
        }
        summary.setExerciseTypeBreakdown(breakdown);

        // Quality score
        double quality = Math.min(1.0, records.size() / 7.0); // 1 session/day = ideal
        summary.setQualityScore(quality);

        return summary;
```

```
    }

    static long computeNextMonday(long currentTimeMs) {
        ZonedDateTime now = Instant.ofEpochMilli(currentTimeMs).atZone(ZoneOffset.UTC);
        ZonedDateTime nextMonday = now.toLocalDate()
                .with(TemporalAdjusters.next(DayOfWeek.MONDAY))
                .atStartOfDay(ZoneOffset.UTC);
        return nextMonday.toInstant().toEpochMilli();
    }
}
```

- [ ] **Step 2: Compile**

Run: `cd backend/shared-infrastructure/flink-processing && mvn compile -pl . -q 2>&1 | tail -5`
Expected: BUILD SUCCESS

- [ ] **Step 3: Commit**

```
git add backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/operators/Module
git commit -m "feat(module11b): add Module11b_FitnessPatternAggregator weekly KPF with VO2max trending"
```

Task 12: Wire Module 11/11b into FlinkJobOrchestrator

Files: - Modify: `FlinkJobOrchestrator.java`

- [ ] **Step 1: Add imports**

```
import com.cardiofit.flink.models.ActivityResponseRecord;
import com.cardiofit.flink.models.FitnessPatternSummary;
```

- [ ] **Step 2: Add switch cases for module11 and module11b**

In the `main()` method switch statement, add before the `default:` case:

```
            case "activity-response":
            case "module11":
            case "activity-response-correlator":
                launchActivityResponseCorrelator(env);
                break;

            case "fitness-patterns":
            case "module11b":
            case "fitness-pattern-aggregator":
```

```
                launchFitnessPatternAggregator(env);
                break;
```

- [ ] **Step 3: Add launchActivityResponseCorrelator method**

```
    /**
     * Module 11: Activity Response Correlator.
     * Exercise-session-window-driven per-activity HR/glucose/BP correlation.
     * Single sink: ActivityResponseRecord → flink.activity-response.
     */
    private static void launchActivityResponseCorrelator(StreamExecutionEnvironment env) {
        LOG.info("Launching Module 11: Activity Response Correlator pipeline");

        String bootstrap = KafkaConfigLoader.getBootstrapServers();

        KafkaSource<CanonicalEvent> source = KafkaSource.<CanonicalEvent>builder()
                .setBootstrapServers(bootstrap)
                .setTopics(KafkaTopics.ENRICHED_PATIENT_EVENTS.getTopicName())
                .setGroupId("flink-module11-activity-response-v1")
                .setStartingOffsets(OffsetsInitializer.earliest())
                .setValueOnlyDeserializer(new CanonicalEventDeserializer())
                .build();

        SingleOutputStreamOperator<ActivityResponseRecord> records = env
                .fromSource(source,
                        WatermarkStrategy.<CanonicalEvent>forBoundedOutOfOrderness(Duration.ofMinutes(2))
                                .withTimestampAssigner((e, ts) -> e.getEventTime()),
                        "Kafka Source: Enriched Patient Events (Module 11)")
                .keyBy(CanonicalEvent::getPatientId)
                .process(new Module11_ActivityResponseCorrelator())
                .uid("module11-activity-response-correlator")
                .name("Module 11: Activity Response Correlator");

        records.sinkTo(
                KafkaSink.<ActivityResponseRecord>builder()
                        .setBootstrapServers(bootstrap)
                        .setDeliveryGuarantee(DeliveryGuarantee.AT_LEAST_ONCE)
                        .setTransactionalIdPrefix("m11-activity-response")
                        .setRecordSerializer(
                                KafkaRecordSerializationSchema.<ActivityResponseRecord>builder()
                                        .setTopic(KafkaTopics.FLINK_ACTIVITY_RESPONSE.getTopicName())
                                        .setValueSerializationSchema(new JsonSerializer<ActivityResponseR
                                        .build())
                        .build()
        ).name("Sink: Activity Response Records");

        LOG.info("Module 11 pipeline configured: source=[{}], sink=[{}]",
                KafkaTopics.ENRICHED_PATIENT_EVENTS.getTopicName(),
                KafkaTopics.FLINK_ACTIVITY_RESPONSE.getTopicName());
    }
```

- [ ] **Step 4: Add launchFitnessPatternAggregator method**

```
    /**
     * Module 11b: Fitness Pattern Aggregator.
     * Weekly aggregation of activity response records with VO2max estimation.
     * Separate job from Module 11 for failure isolation.
     */
    private static void launchFitnessPatternAggregator(StreamExecutionEnvironment env) {
        LOG.info("Launching Module 11b: Fitness Pattern Aggregator pipeline");

        String bootstrap = KafkaConfigLoader.getBootstrapServers();

        KafkaSource<ActivityResponseRecord> source = KafkaSource.<ActivityResponseRecord>builder()
                .setBootstrapServers(bootstrap)
                .setTopics(KafkaTopics.FLINK_ACTIVITY_RESPONSE.getTopicName())
                .setGroupId("flink-module11b-fitness-patterns-v1")
                .setStartingOffsets(OffsetsInitializer.earliest())
                .setValueOnlyDeserializer(new ActivityResponseRecordDeserializer())
                .build();

        SingleOutputStreamOperator<FitnessPatternSummary> summaries = env
                .fromSource(source,
                        WatermarkStrategy.<ActivityResponseRecord>forBoundedOutOfOrderness(Duration.ofMin
                                .withTimestampAssigner((r, ts) -> r.getActivityStartTime()),
                        "Kafka Source: Activity Response Records (Module 11b)")
                .keyBy(ActivityResponseRecord::getPatientId)
                .process(new Module11b_FitnessPatternAggregator())
                .uid("module11b-fitness-pattern-aggregator")
                .name("Module 11b: Fitness Pattern Aggregator");

        summaries.sinkTo(
                KafkaSink.<FitnessPatternSummary>builder()
                        .setBootstrapServers(bootstrap)
                        .setDeliveryGuarantee(DeliveryGuarantee.AT_LEAST_ONCE)
                        .setTransactionalIdPrefix("m11b-fitness-patterns")
                        .setRecordSerializer(
                                KafkaRecordSerializationSchema.<FitnessPatternSummary>builder()
                                        .setTopic(KafkaTopics.FLINK_FITNESS_PATTERNS.getTopicName())
                                        .setValueSerializationSchema(new JsonSerializer<FitnessPatternSum
                                        .build())
                        .build()
        ).name("Sink: Fitness Pattern Summaries");

        LOG.info("Module 11b pipeline configured: source=[{}], sink=[{}]",
                KafkaTopics.FLINK_ACTIVITY_RESPONSE.getTopicName(),
                KafkaTopics.FLINK_FITNESS_PATTERNS.getTopicName());
    }

    /** Deserializes JSON bytes into an ActivityResponseRecord using Jackson. */
    static class ActivityResponseRecordDeserializer implements DeserializationSchema<ActivityResponseReco
        private transient ObjectMapper mapper;

        @Override
        public void open(InitializationContext context) {
            mapper = new ObjectMapper();
```

```
            mapper.registerModule(new JavaTimeModule());
        }

        @Override
        public ActivityResponseRecord deserialize(byte[] message) throws IOException {
            if (mapper == null) {
                mapper = new ObjectMapper();
                mapper.registerModule(new JavaTimeModule());
            }
            if (message == null || message.length == 0) return null;
            return mapper.readValue(message, ActivityResponseRecord.class);
        }

        @Override
        public boolean isEndOfStream(ActivityResponseRecord nextElement) {
            return false;
        }

        @Override
        public TypeInformation<ActivityResponseRecord> getProducedType() {
            return TypeInformation.of(ActivityResponseRecord.class);
        }
    }
```

- [ ] **Step 5: Add Module 11/11b to launchFullPipeline**

```
        // Module 11: Activity Response Correlator
        LOG.info("Initializing Module 11: Activity Response Correlator");
        launchActivityResponseCorrelator(env);

        // Module 11b: Fitness Pattern Aggregator
        LOG.info("Initializing Module 11b: Fitness Pattern Aggregator");
        launchFitnessPatternAggregator(env);
```

- [ ] **Step 6: Add KafkaTopics constants**

In `KafkaTopics.java` enum, add:

```
    FLINK_ACTIVITY_RESPONSE("flink.activity-response"),
    FLINK_FITNESS_PATTERNS("flink.fitness-patterns"),
```

- [ ] **Step 7: Compile**

Run: `cd backend/shared-infrastructure/flink-processing && mvn compile -pl . -q 2>&1 | tail -5`
Expected: BUILD SUCCESS

- [ ] **Step 8: Commit**

```
git add \
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/FlinkJobOrchestrator.j
  backend/shared-infrastructure/flink-processing/src/main/java/com/cardiofit/flink/models/KafkaTopics.jav
git commit -m "feat(module11): wire Module 11 and 11b into FlinkJobOrchestrator with lazy-init deserializ
```

Task 13: Create Module11TestBuilder and Integration Tests

Files: - Create: `builders/Module11TestBuilder.java` - Create: `operators/Module11SessionWindowTest.java` - Create: `operators/Module11ConcurrentActivityTest.java` - Create: `operators/Module11IntensityZoneTest.java`

- [ ] **Step 1: Create Module11TestBuilder**

```
package com.cardiofit.flink.builders;

import com.cardiofit.flink.models.*;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;

/**
 * Test builder for Module 11/11b tests.
 */
public class Module11TestBuilder {

    private static final long HOUR_MS = 3_600_000L;
    private static final long MIN_MS = 60_000L;

    public static final long BASE_TIME = 1743552000000L; // 2025-04-02 00:00:00 UTC

    public static long minutesAfter(long base, int minutes) { return base + minutes * MIN_MS; }
    public static long hoursAfter(long base, int hours) { return base + hours * HOUR_MS; }

    // --- Activity Events ---
    public static CanonicalEvent activityEvent(String patientId, long timestamp,
                                               String exerciseType, int durationMinutes) {
        CanonicalEvent event = baseEvent(patientId, EventType.PATIENT_REPORTED, timestamp);
        Map<String, Object> payload = new HashMap<>();
        payload.put("report_type", "ACTIVITY_LOG");
        payload.put("exercise_type", exerciseType);
        payload.put("duration_minutes", durationMinutes);
        event.setPayload(payload);
        return event;
    }

    public static CanonicalEvent activityWithMETs(String patientId, long timestamp,
                                                  String exerciseType, int durationMinutes, double mets)
        CanonicalEvent event = activityEvent(patientId, timestamp, exerciseType, durationMinutes);
        event.getPayload().put("mets", mets);
```

```
        return event;
    }

    // --- HR Events ---
    public static CanonicalEvent hrReading(String patientId, long timestamp, double heartRate) {
        CanonicalEvent event = baseEvent(patientId, EventType.DEVICE_READING, timestamp);
        Map<String, Object> payload = new HashMap<>();
        payload.put("heart_rate", heartRate);
        payload.put("source", "WEARABLE");
        event.setPayload(payload);
        return event;
    }

    public static CanonicalEvent restingHRReading(String patientId, long timestamp, double heartRate) {
        CanonicalEvent event = hrReading(patientId, timestamp, heartRate);
        event.getPayload().put("activity_state", "RESTING");
        return event;
    }

    // --- Glucose Events (reusing Module 10 patterns) ---
    public static CanonicalEvent cgmReading(String patientId, long timestamp, double glucose) {
        CanonicalEvent event = baseEvent(patientId, EventType.DEVICE_READING, timestamp);
        Map<String, Object> payload = new HashMap<>();
        payload.put("glucose_value", glucose);
        payload.put("source", "CGM");
        event.setPayload(payload);
        return event;
    }

    // --- BP Events ---
    public static CanonicalEvent bpReading(String patientId, long timestamp,
                                           double sbp, double dbp) {
        CanonicalEvent event = baseEvent(patientId, EventType.VITAL_SIGN, timestamp);
        Map<String, Object> payload = new HashMap<>();
        payload.put("systolic_bp", sbp);
        payload.put("diastolic_bp", dbp);
        event.setPayload(payload);
        return event;
    }

    // --- State Builders ---
    public static ActivityCorrelationState emptyState(String patientId) {
        return new ActivityCorrelationState(patientId);
    }

    public static ActivityCorrelationState stateWithAge(String patientId, int age) {
        ActivityCorrelationState state = new ActivityCorrelationState(patientId);
        state.setPatientAge(age);
        return state;
    }

    // --- ActivityResponseRecord Builder ---
    public static ActivityResponseRecord activityRecord(String patientId, long timestamp,
                                                        ExerciseType type, double mets, double durationM
```

```
                                                                Double peakHR, Double hrr1, Double glucoseDelta)
        return ActivityResponseRecord.builder()
                .recordId("test-" + UUID.randomUUID())
                .patientId(patientId)
                .activityEventId("act-" + UUID.randomUUID())
                .activityStartTime(timestamp)
                .exerciseType(type)
                .reportedMETs(mets)
                .activityDurationMin(durationMin)
                .metMinutes(mets * durationMin)
                .peakHR(peakHR)
                .hrr1(hrr1)
                .hrRecoveryClass(hrr1 != null ? HRRecoveryClass.fromHRR1(hrr1) : HRRecoveryClass.INSUFFIC
                .exerciseGlucoseDelta(glucoseDelta)
                .qualityScore(0.7)
                .build();
    }

    private static CanonicalEvent baseEvent(String patientId, EventType type, long timestamp) {
        CanonicalEvent event = new CanonicalEvent();
        event.setId(UUID.randomUUID().toString());
        event.setPatientId(patientId);
        event.setEventType(type);
        event.setEventTime(timestamp);
        event.setProcessingTime(System.currentTimeMillis());
        event.setCorrelationId("test-" + UUID.randomUUID());
        return event;
    }
}
```

- [ ] **Step 2: Create Module11SessionWindowTest**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.builders.Module11TestBuilder;
import com.cardiofit.flink.models.*;
import org.junit.jupiter.api.Test;
import java.util.HashMap;
import java.util.Map;
import static org.junit.jupiter.api.Assertions.*;

class Module11SessionWindowTest {

    private static final long BASE = Module11TestBuilder.BASE_TIME;
    private static final long MIN = 60_000L;
    private static final long HOUR = 3_600_000L;

    @Test
    void openSession_timerAt_durationPlus2hPlus5m() {
        ActivityCorrelationState state = Module11TestBuilder.emptyState("P1");
        Map<String, Object> payload = new HashMap<>();
        payload.put("exercise_type", "AEROBIC");
        long timerFireTime = state.openSession("act-1", BASE, 45 * MIN, payload);
```

```
        // 45 min activity + 2h recovery + 5 min grace = 2h50m
        long expected = BASE + 45 * MIN + 2 * HOUR + 5 * MIN;
        assertEquals(expected, timerFireTime);
    }

    @Test
    void hrReadings_addedDuringExerciseAndRecovery() {
        ActivityCorrelationState state = Module11TestBuilder.emptyState("P1");
        state.openSession("act-1", BASE, 30 * MIN, new HashMap<>());

        // HR during exercise
        state.addHRReading(BASE + 10 * MIN, 140.0, "WEARABLE");
        state.addHRReading(BASE + 20 * MIN, 155.0, "WEARABLE");
        // HR during recovery (after 30 min activity end)
        state.addHRReading(BASE + 35 * MIN, 130.0, "WEARABLE");
        state.addHRReading(BASE + 60 * MIN, 100.0, "WEARABLE");

        assertEquals(4, state.getActiveSessions().get("act-1").hrWindow.size());
    }

    @Test
    void hrReadings_ignoredOutsideWindow() {
        ActivityCorrelationState state = Module11TestBuilder.emptyState("P1");
        state.openSession("act-1", BASE, 30 * MIN, new HashMap<>());

        // HR 3 hours after exercise end → outside 2h recovery window
        state.addHRReading(BASE + 30 * MIN + 3 * HOUR, 75.0, "WEARABLE");

        assertEquals(0, state.getActiveSessions().get("act-1").hrWindow.size());
    }

    @Test
    void preExerciseBP_retroactiveAttachment() {
        ActivityCorrelationState state = Module11TestBuilder.emptyState("P1");
        // BP 20 min before exercise (within 30 min lookback)
        state.addBPReading(BASE - 20 * MIN, 125.0, 82.0);
        state.openSession("act-1", BASE, 30 * MIN, new HashMap<>());

        ActivityCorrelationState.ActivitySession session = state.getActiveSessions().get("act-1");
        assertTrue(session.bpWindow.hasPreMeal()); // reusing BPWindow: pre-meal = pre-exercise
        assertEquals(125.0, session.bpWindow.getPreMealSBP());
    }

    @Test
    void peakExerciseBP_trackedDuringActivity() {
        ActivityCorrelationState state = Module11TestBuilder.emptyState("P1");
        state.openSession("act-1", BASE, 30 * MIN, new HashMap<>());

        state.addBPReading(BASE + 10 * MIN, 155.0, 88.0);
        state.addBPReading(BASE + 20 * MIN, 170.0, 92.0); // higher → peak
        state.addBPReading(BASE + 25 * MIN, 160.0, 90.0);

        ActivityCorrelationState.ActivitySession session = state.getActiveSessions().get("act-1");
        assertEquals(170.0, session.peakExerciseSBP);
```

```
    }

    @Test
    void durationCap_maxAt4hours() {
        ActivityCorrelationState state = Module11TestBuilder.emptyState("P1");
        // Report 6h activity → should be capped at 4h
        long timerFireTime = state.openSession("act-1", BASE, 6 * HOUR, new HashMap<>());
        long expected = BASE + 4 * HOUR + 2 * HOUR + 5 * MIN; // capped at 4h + 2h + 5m
        assertEquals(expected, timerFireTime);
    }

    @Test
    void closeSession_removesFromActive() {
        ActivityCorrelationState state = Module11TestBuilder.emptyState("P1");
        state.openSession("act-1", BASE, 30 * MIN, new HashMap<>());
        assertEquals(1, state.getActiveSessions().size());

        ActivityCorrelationState.ActivitySession closed = state.closeSession("act-1");
        assertNotNull(closed);
        assertEquals(0, state.getActiveSessions().size());
    }
}
```

- [ ] **Step 3: Create Module11ConcurrentActivityTest**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.builders.Module11TestBuilder;
import com.cardiofit.flink.models.*;
import org.junit.jupiter.api.Test;
import java.util.HashMap;
import static org.junit.jupiter.api.Assertions.*;

class Module11ConcurrentActivityTest {

    private static final long BASE = Module11TestBuilder.BASE_TIME;
    private static final long MIN = 60_000L;

    @Test
    void twoActivitiesWithin30min_bothFlagged() {
        ActivityCorrelationState state = Module11TestBuilder.emptyState("P1");
        state.openSession("act-1", BASE, 30 * MIN, new HashMap<>());
        state.openSession("act-2", BASE + 20 * MIN, 30 * MIN, new HashMap<>());

        assertTrue(state.getActiveSessions().get("act-1").concurrent);
        assertTrue(state.getActiveSessions().get("act-2").concurrent);
    }

    @Test
    void twoActivitiesOver30minApart_notFlagged() {
        ActivityCorrelationState state = Module11TestBuilder.emptyState("P1");
        state.openSession("act-1", BASE, 30 * MIN, new HashMap<>());
        state.openSession("act-2", BASE + 45 * MIN, 30 * MIN, new HashMap<>());
```

```
        assertFalse(state.getActiveSessions().get("act-1").concurrent);
        assertFalse(state.getActiveSessions().get("act-2").concurrent);
    }

    @Test
    void hrReadings_feedBothConcurrentSessions() {
        ActivityCorrelationState state = Module11TestBuilder.emptyState("P1");
        state.openSession("act-1", BASE, 60 * MIN, new HashMap<>());
        state.openSession("act-2", BASE + 20 * MIN, 60 * MIN, new HashMap<>());

        // HR reading at 30 min — within both windows
        state.addHRReading(BASE + 30 * MIN, 155.0, "WEARABLE");

        assertEquals(1, state.getActiveSessions().get("act-1").hrWindow.size());
        assertEquals(1, state.getActiveSessions().get("act-2").hrWindow.size());
    }
}
```

- [ ] **Step 4: Create Module11IntensityZoneTest**

```
package com.cardiofit.flink.operators;

import com.cardiofit.flink.models.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class Module11IntensityZoneTest {

    @Test
    void zoneClassification_allZones() {
        double hrMax = 180.0; // age ~40

        assertEquals(ActivityIntensityZone.ZONE_1_RECOVERY,
                ActivityIntensityZone.fromHeartRate(95, hrMax));  // 53%
        assertEquals(ActivityIntensityZone.ZONE_2_AEROBIC,
                ActivityIntensityZone.fromHeartRate(115, hrMax)); // 64%
        assertEquals(ActivityIntensityZone.ZONE_3_TEMPO,
                ActivityIntensityZone.fromHeartRate(135, hrMax)); // 75%
        assertEquals(ActivityIntensityZone.ZONE_4_THRESHOLD,
                ActivityIntensityZone.fromHeartRate(155, hrMax)); // 86%
        assertEquals(ActivityIntensityZone.ZONE_5_ANAEROBIC,
                ActivityIntensityZone.fromHeartRate(170, hrMax)); // 94%
    }

    @Test
    void tanakaFormula_correctForDifferentAges() {
        // Age 20: 208 - 0.7*20 = 194
        assertEquals(194.0, ActivityIntensityZone.estimateHRMax(20), 0.1);
        // Age 40: 208 - 0.7*40 = 180
        assertEquals(180.0, ActivityIntensityZone.estimateHRMax(40), 0.1);
        // Age 60: 208 - 0.7*60 = 166
        assertEquals(166.0, ActivityIntensityZone.estimateHRMax(60), 0.1);
```

```
    }

    @Test
    void isHighIntensity_zone4and5() {
        assertTrue(ActivityIntensityZone.ZONE_4_THRESHOLD.isHighIntensity());
        assertTrue(ActivityIntensityZone.ZONE_5_ANAEROBIC.isHighIntensity());
        assertFalse(ActivityIntensityZone.ZONE_3_TEMPO.isHighIntensity());
        assertFalse(ActivityIntensityZone.ZONE_1_RECOVERY.isHighIntensity());
    }

    @Test
    void exerciseType_fromString_variants() {
        assertEquals(ExerciseType.AEROBIC, ExerciseType.fromString("running"));
        assertEquals(ExerciseType.AEROBIC, ExerciseType.fromString("CYCLING"));
        assertEquals(ExerciseType.RESISTANCE, ExerciseType.fromString("weights"));
        assertEquals(ExerciseType.HIIT, ExerciseType.fromString("interval"));
        assertEquals(ExerciseType.FLEXIBILITY, ExerciseType.fromString("yoga"));
        assertEquals(ExerciseType.MIXED, ExerciseType.fromString(null));
        assertEquals(ExerciseType.MIXED, ExerciseType.fromString("unknown_activity"));
    }

    @Test
    void fitnessLevel_fromVO2max() {
        assertEquals(FitnessLevel.EXCELLENT, FitnessLevel.fromVO2max(48.0));
        assertEquals(FitnessLevel.GOOD, FitnessLevel.fromVO2max(38.0));
        assertEquals(FitnessLevel.AVERAGE, FitnessLevel.fromVO2max(28.0));
        assertEquals(FitnessLevel.BELOW_AVERAGE, FitnessLevel.fromVO2max(20.0));
        assertEquals(FitnessLevel.POOR, FitnessLevel.fromVO2max(15.0));
    }

    @Test
    void hrRecoveryClass_fromHRR1() {
        assertEquals(HRRecoveryClass.EXCELLENT, HRRecoveryClass.fromHRR1(30.0));
        assertEquals(HRRecoveryClass.NORMAL, HRRecoveryClass.fromHRR1(20.0));
        assertEquals(HRRecoveryClass.BLUNTED, HRRecoveryClass.fromHRR1(14.0));
        assertEquals(HRRecoveryClass.ABNORMAL, HRRecoveryClass.fromHRR1(8.0));
    }

    @Test
    void exerciseBPResponse_classifications() {
        assertEquals(ExerciseBPResponse.NORMAL,
                ExerciseBPResponse.classify(120.0, 160.0, 122.0));
        assertEquals(ExerciseBPResponse.EXAGGERATED,
                ExerciseBPResponse.classify(120.0, 215.0, 130.0));
        assertEquals(ExerciseBPResponse.POST_EXERCISE_HYPOTENSION,
                ExerciseBPResponse.classify(140.0, 170.0, 130.0));
        assertEquals(ExerciseBPResponse.HYPOTENSIVE_RESPONSE,
                ExerciseBPResponse.classify(140.0, 170.0, 115.0));
        assertEquals(ExerciseBPResponse.INCOMPLETE,
                ExerciseBPResponse.classify(null, 160.0, 130.0));
```

```
    }
}
```

- [ ] **Step 5: Run all tests**

Run: `cd backend/shared-infrastructure/flink-processing && mvn test -pl . -Dtest="Module11SessionWindowTest,Module11ConcurrentActivityTest,Module11IntensityZoneTest" -q 2>&1 | tail -15` Expected: PASS (all tests)

- [ ] **Step 6: Commit**

```
git add \
  backend/shared-infrastructure/flink-processing/src/test/java/com/cardiofit/flink/builders/Module11TestB
  backend/shared-infrastructure/flink-processing/src/test/java/com/cardiofit/flink/operators/Module11Sess
  backend/shared-infrastructure/flink-processing/src/test/java/com/cardiofit/flink/operators/Module11Conc
  backend/shared-infrastructure/flink-processing/src/test/java/com/cardiofit/flink/operators/Module11Inte
git commit -m "feat(module11): add session window, concurrent activity, intensity zone tests + test build
```

Self-Review

Spec Coverage

- Exercise session windows with activity_duration + 2h recovery + 5 min grace ✅
- Duration cap at 4h activity max ✅
- Processing-time timer (same trade-off as Module 10 — documented) ✅
- Three-phase HR analysis (pre-exercise, active, recovery) ✅
- HRR_1/HRR_2 with ±30s tolerance and clinical classification (Cole et al.) ✅
- Five-zone HR distribution (ACSM-aligned, Tanaka HR_max) ✅
- Exercise glucose delta with aerobic drop vs HIIT spike distinction ✅
- Hypoglycemia flag (< 70 mg/dL) ✅
- Rebound hyperglycemia flag (> baseline + 40 mg/dL post-exercise) ✅
- Exercise BP response: NORMAL / EXAGGERATED / PEH / HYPOTENSIVE / INCOMPLETE ✅
- Rate-Pressure Product (HR × SBP) ✅
- Pre-exercise BP retroactive buffer (30 min lookback) ✅
- Peak exercise BP tracking during active phase ✅
- Concurrent activity detection (30 min threshold) ✅
- MET-minutes dose calculation with WHO benchmark ✅
- Submaximal VO2max estimation (Astrand-Ryhming hybrid) ✅
- 90-day VO2max trend via linear regression ✅
- Fitness level classification (ACSM-based, 5 tiers) ✅

- Per-exercise-type weekly breakdown ✅
- ActivityResponseRecord (32 fields) ✅
- FitnessPatternSummary (24 fields) ✅
- Two-job architecture (Module 11 + 11b separate Flink jobs) ✅
- Orchestrator wiring with ActivityResponseRecordDeserializer (lazy-init guard) ✅
- KafkaTopics constants for new topics ✅

Type Consistency

- `ActivityResponseRecord` used consistently across Module 11 output, Module 11b input, test builder
- `HRWindow.HRReading` fields: `timestamp`, `heartRate`, `source` — consistent
- `HRRecoveryClass.fromHRR1()` signature matches analyzer usage
- `ActivityCorrelationState.ActivitySession` fields match KPF access patterns
- `Module11HRAnalyzer.Result` fields match `ActivityResponseRecord.Builder` calls
- `GlucoseWindow` reused from Module 10 for glucose readings (no duplication)
- `BPWindow` reused from Module 10 for pre/post BP (pre-meal ≡ pre-exercise)

Improvements Over Module 10 Patterns (incorporating your review feedback)

1. **Window duration is deterministic**: `timerFireTime - activityStartTime` instead of `System.currentTimeMillis()`
2. **Deserializer has lazy-init guard**: Null-safe `mapper` initialization in `deserialize()`
3. **KafkaTopics constants explicitly created**: FLINK_ACTIVITY_RESPONSE, FLINK_FITNESS_PATTERNS
4. **Duration capped**: MAX_ACTIVITY_DURATION_MS = 4h prevents runaway timers
5. **Concurrent activity detection** uses 30 min threshold (tighter than Module 10's 90 min overlap, matching exercise domain)

Test Coverage (~48 tests across 9 classes)

- Module11HRAnalyzerTest: 6 tests
- Module11GlucoseExerciseAnalyzerTest: 5 tests
- Module11ExerciseBPAnalyzerTest: 7 tests
- Module11SessionWindowTest: 7 tests
- Module11ConcurrentActivityTest: 3 tests
- Module11IntensityZoneTest: 7 tests (covers all enum classifications)
- Module11bVO2maxEstimatorTest: 6 tests
- Module11bExerciseDoseCalculatorTest: 4 tests
- **Total: 45 unit tests** (remaining ~3 would be KPF integration tests for the full pipeline)